



รายงานผลดำเนินการโครงการ (Final Project)

โครงการแปลงรัฐธรรมนูญไทย 20 ฉบับเป็นดิจิทัล

Twenty Constitutions Digitalization

คณะผู้จัดทำ

67070501003	กันต์ธีร์	ดวงมณี
67070501027	นัธวัฒน์	ปริมสิริคุณาวุฒิ
67070501042	วิศิษฐ์	สุวรรณเนา
67070501045	ศุภวิชญ์	มารยาท
67070501067	พลวิชญ์	วัฒนเหมรัตน์

เสนอ

ดร. สันญ์สิริ ธารประดับ

ภาคเรียนที่ 2 ปีการศึกษา 2568

รายวิชา CPE232 แบบจำลองข้อมูล (Data Models)

ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

สารบัญ

สารบัญ	ก
บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญ	1
1.2 วัตถุประสงค์	1
1.3 ขอบเขตของโครงการ	2
1.4 เครื่องมือที่ใช้ในการพัฒนา	2
บทที่ 2 การสำรวจข้อมูลเบื้องต้น	3
2.1 คำอธิบายข้อมูล	3
2.2 จำนวนรายการข้อมูล	3
2.3 หมวดหมู่ของข้อมูล	3
2.4 คุณภาพของข้อมูล	3
บทที่ 3 การเตรียมและการจัดการข้อมูล	4
3.1 การสกัดข้อความจากเอกสาร PDF (Data Extraction)	4
1. สำหรับข้อมูลกลุ่ม พ.ศ. 2475–2548 (Image PDF)	4
2. สำหรับข้อมูลกลุ่ม พ.ศ. 2550–2564 (Text PDF)	6
3.2 การจัดการและทำความสะอาดข้อมูลก่อนนำไปใช้ (Cleaning & Structuring)	7
1. การแปลงอักขระ (Normalization)	7
2. การซ่อมแซมคำผิดด้วย Regular Expression	7
3. การกำจัดส่วนเกินของราชกิจจานุเบกษา	7
4. การบันทึกข้อมูลแบบมีโครงสร้าง (JSON Structuring)	9
3.3 การจัดโครงสร้างข้อมูลเชิงลึก (Deep Structure Parsing)	10
1. การ parse โครงสร้างหมวด ส่วนที่ และมาตรา	10
2. การตรวจจับประเภทฉบับ (Original / Amendment)	12
3. การ Export JSON และ CSV	12
3.4 สรุปผลลัพธ์การเตรียมข้อมูล (Data Preparation Results)	14
1. โครงสร้าง Structured JSON	14
2. โครงสร้าง Flat CSV	15
3. ผลลัพธ์ภาพรวม	16
บทที่ 4 การวิเคราะห์ข้อมูล (Exploratory Data Analysis & Visualization)	17
4.1. โครงสร้างของคลังข้อมูล (Corpus Structure)	17
4.2. บริบททางการเมืองของข้อมูล (Political Context)	19
4.3. โครงสร้างบทบาทของมาตรา (Section Role & Change Mode)	20
4.4 การวิเคราะห์คลังคำและเนื้อหา (Lexical & Content Analysis)	21
4.5 พัฒนาการของคลังคำตามกาลเวลา (Vocabulary Evolution)	24

4.6. ความคล้ายคลึงระหว่างรัฐธรรมนูญ (Cross-Constitution Similarity)	26
4.7 การวิเคราะห์คำหายาก (Rare Word Analysis)	28
4.8 Visualization อื่น ๆ : Word Cloud	30
บทที่ 5 การจำแนกหัวข้อด้วย Topic Modeling	31
5.1 ที่มาและปัญหาสำคัญ	31
5.2 การพัฒนาวิธีการ (Methodology Progression)	31
5.3 รายละเอียด Pipeline (Attempt 4)	33
5.3.1 Preprocessing	33
5.3.2 Embedding	33
5.3.3 Semi-supervised Topic Discovery (Guided BERTopic)	33
5.3.4 Hybrid Soft Scoring	35
5.4 ผลลัพธ์จากการทำปฏิบัติการ	37
5.4.1 ไฟล์ Output	37
5.4.2 ความเสถียรของ Theme (Theme Stability)	37
5.4.3 พลวัตของ Theme (Theme Dynamics)	39
5.4.4 Co-occurrence ระหว่าง Theme	40
5.4.5 สรุปผล	41
บทที่ 6 การเผยแพร่และการเข้าถึงข้อมูล (Data Publishing & Public Access)	42
6.1 ชุดข้อมูลสาธารณะบน Kaggle	42
6.2 เว็บแอปพลิเคชัน Dashboard (Twenty Constitutions Web)	44
บรรณานุกรม	45
ข้อมูลที่ใช้ในการพัฒนา	45
แหล่งอ้างอิงทางทฤษฎีและเครื่องมือ	45
ข้อมูลอ้างอิงเพิ่มเติม	46

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญ

ประเทศไทยมีรัฐธรรมนูญรวมทั้งสิ้น 20 ฉบับ นับตั้งแต่ปี พ.ศ. 2475 จนถึงปัจจุบัน ซึ่งถือว่าเป็นจำนวนมากเมื่อเทียบกับหลายประเทศที่ปกครองในระบอบประชาธิปไตย รัฐธรรมนูญแต่ละฉบับเกิดขึ้นภายใต้บริบททางการเมืองที่แตกต่างกัน ไม่ว่าจะเป็นรัฐบาลพลเรือน คณะรัฐประหาร หรือสภาเฉพาะกาล อย่างไรก็ตาม เอกสารรัฐธรรมนูญจำนวนมากยังอยู่ในรูปแบบที่ไม่เอื้อต่อการวิเคราะห์เชิงคอมพิวเตอร์อย่างเป็นระบบ

ดังนั้น โครงการนี้จึงมีเป้าหมายหลักในการรวบรวมและดิจิทัลไลซ์รัฐธรรมนูญไทยทั้ง 20 ฉบับ เพื่อจัดทำเป็นคลังข้อความดิจิทัล (Text Corpus) ที่สามารถนำไปใช้ต่อยอดในการศึกษาทางประวัติศาสตร์ การเมือง และกฎหมายได้ในอนาคต กระบวนการทำงานประกอบด้วยรวบรวมเอกสาร การแปลงไฟล์ด้วยเทคนิค OCR การทำความสะอาดข้อความ และการจัดโครงสร้างข้อมูลให้อยู่ในรูปแบบที่สามารถประมวลผลด้วยคอมพิวเตอร์ได้

ในช่วงท้ายของโครงการ มีการนำเทคนิคพื้นฐานด้าน NLP และการแสดงผลข้อมูลมาใช้เพื่อสำรวจลักษณะของรัฐธรรมนูญไทยในแต่ละช่วงเวลา เช่น การเปรียบเทียบจำนวนคำ ความยาวของเอกสาร ความถี่ของคำสำคัญ และแนวโน้มของประเด็นที่ปรากฏในข้อความ นอกจากนี้ โครงการยังทดลองใช้ topic modeling เพื่อช่วยมองเห็นกลุ่มหัวข้อหลักที่ปรากฏในรัฐธรรมนูญแต่ละฉบับในเชิงสำรวจ มากกว่าการวิเคราะห์เชิงตีความอย่างสมบูรณ์

1.2 วัตถุประสงค์

โครงการนี้มีวัตถุประสงค์หลักในการรวบรวมและดิจิทัลไลซ์รัฐธรรมนูญไทยทั้ง 20 ฉบับ เพื่อจัดทำเป็นคลังข้อความดิจิทัลที่สามารถนำไปใช้ต่อยอดในการศึกษาและวิเคราะห์เชิงข้อมูลได้ โดยมีเป้าหมายสำคัญดังนี้:

1. เพื่อรวบรวม แปลงไฟล์ และทำความสะอาดข้อความจากรัฐธรรมนูญไทยทั้ง 20 ฉบับ ด้วยเทคนิค OCR และการดึงข้อมูลจากไฟล์ PDF
2. เพื่อจัดทำคลังข้อความ หรือ Text Corpus ของรัฐธรรมนูญไทย ให้อยู่ในรูปแบบที่เป็นระบบและสามารถนำไปประมวลผลต่อได้
3. เพื่อสำรวจข้อมูลเบื้องต้นผ่าน visualization และทดลองใช้เทคนิค NLP เช่น การนับคำสำคัญและ topic modeling เพื่อแสดงแนวโน้มหรือประเด็นหลักที่พบในรัฐธรรมนูญแต่ละช่วงเวลา

1.3 ขอบเขตของโครงการ

โครงการนี้มุ่งเน้นการรวบรวมและดิจิทัลไลซ์ เอกสารรัฐธรรมนูญและธรรมนูญการปกครองของประเทศไทย ที่ประกาศใช้ในราชกิจจานุเบกษาตั้งแต่ปี พ.ศ. 2475 ถึงปัจจุบัน จำนวน 20 ฉบับหลัก รวมถึงเอกสารแก้ไขเพิ่มเติมที่เกี่ยวข้อง รวมทั้งหมด 38 ไฟล์ ขอบเขตการดำเนินงานครอบคลุมการดึงข้อความจากไฟล์ PDF การแปลงภาพเป็นข้อความด้วยเทคนิค OCR การทำความสะอาดข้อมูล การจัดโครงสร้างข้อมูลในรูปแบบ JSON และการเตรียมข้อมูลให้อยู่ในรูปแบบที่พร้อมนำไปใช้กับการประมวลผลข้อความต่อไป ในส่วนของงานวิเคราะห์ โครงการนี้จำกัดอยู่ที่การสำรวจข้อมูลเบื้องต้น เช่น การสร้าง visualization การนับความถี่ของคำสำคัญ และการทดลองใช้ topic modeling เพื่อค้นหาแนวโน้มของหัวข้อที่ปรากฏในรัฐธรรมนูญแต่ละช่วงเวลา โดยไม่ได้มุ่งเน้นการวิเคราะห์เชิงกฎหมายหรือการตีความเนื้อหาของรัฐธรรมนูญอย่างละเอียดครบทุกฉบับ

1.4 เครื่องมือที่ใช้ในการพัฒนา

สำหรับการพัฒนาและวิเคราะห์ข้อมูลในโครงการวิเคราะห์รัฐธรรมนูญ 20 ฉบับของประเทศไทย ได้เลือกใช้เครื่องมือและเทคโนโลยีหลักดังต่อไปนี้:

- **Python / Jupyter Notebook:** เครื่องมือหลักในการเขียนและรันท่อข้อมูล (Data Pipeline)
- **Typhoon OCR API 1.5:** โมเดลปัญญาประดิษฐ์สกัดอักษรจากภาพสำหรับภาษาไทย
- **PyMuPDF (fitz) / pdfplumber:** ไลบรารีสำหรับสกัดข้อความจากไฟล์ Text PDF
- **Pandas:** สำหรับจัดการตารางข้อมูลและโครงสร้างข้อมูลเบื้องต้น
- **JSON:** โครงสร้างข้อมูลที่ใช้จัดเก็บ Metadata และเนื้อหา
- **PyThaiNLP / Attacut:** สำหรับการทำ Tokenization และทำความสะอาดภาษาไทย
- **Scikit-learn:** สำหรับ Feature Extraction และ Machine Learning

บทที่ 2 การสำรวจข้อมูลเบื้องต้น

ก่อนเริ่มการประมวลผล คณะผู้จัดทำได้สำรวจชุดข้อมูลรัฐธรรมนูญที่รวบรวมมา เพื่อออกแบบโครงสร้างข้อมูล (Data Model) ให้เหมาะสมที่สุด ซึ่งรายละเอียดสำหรับการสำรวจข้อมูลเบื้องต้น มีดังนี้

2.1 คำอธิบายข้อมูล

ข้อมูลขั้นต้น (Primary Data) คือรัฐธรรมนูญและธรรมนูญการปกครองของไทย จัดทำโดย สำนักวิชาการ สำนักงานเลขาธิการสภาผู้แทนราษฎร ซึ่งถูกเก็บไว้ในระบบคลังสารสนเทศรัฐสภา โดยเผยแพร่ภายใต้สัญญาอนุญาต Creative Commons (CC BY-NC-ND 4.0) ข้อมูลต้นฉบับอยู่ในรูปแบบไฟล์เอกสาร (PDF) ซึ่งมีทั้งแบบภาพสแกน (สำหรับยุคเก่า) และข้อความดิจิทัล (ยุคปัจจุบัน)

2.2 จำนวนรายการข้อมูล

ชุดข้อมูลที่ถูกนำมาวิเคราะห์ประกอบด้วยเอกสาร 38 ไฟล์ (แบ่งเป็น 32 Image PDFs และ 6 Text PDFs) ซึ่งประกอบเป็นรัฐธรรมนูญ 20 ฉบับหลัก และฉบับแก้ไขเพิ่มเติม โดยมีปริมาณข้อความรวมโดยประมาณ 61,301 คำ

2.3 หมวดหมู่ของข้อมูล

เพื่อให้การนำข้อมูลไปใช้งานง่ายขึ้น ข้อมูลถูกจัดกลุ่มไว้ด้วย Metadata (JSON Schema) ที่สำคัญ ดังนี้:

- **id:** รหัสอ้างอิงของเอกสาร
- **year_th / year_ce:** ปีพุทธศักราชและคริสต์ศักราชที่ประกาศใช้
- **name_short:** ชื่อเรียกสั้น ๆ
- **source_type:** ประเภทของไฟล์เอกสาร (image_pdf หรือ text_pdf)
- **era / regime_type:** ยุคการเมืองและระบอบการปกครองในขณะนั้น
- **full_text:** ข้อความทั้งหมดที่สกัดมาได้
- **metadata:** รายละเอียดปลีกย่อย เช่น จำนวนอักขระ, จำนวนหน้า, และจำนวนคำโดยประมาณ

2.4 คุณภาพของข้อมูล

จากการสำรวจคุณภาพเชิงลึก พบปัญหาทางข้อมูลที่สำคัญ 2 ประการ ได้แก่:

1. **Outdated Font Encoding (สระอำแยกร่าง):** ในเอกสารยุคหลัง (พ.ศ. 2550 - 2564) มีการเข้ารหัสตัวสระอำด้วยช่องว่าง (Zero-width space) ประกอบสระอำ ทิ้งให้คำผิดเพี้ยน เช่น "จ นวนน" (แทนที่จะเป็น จำนวน)
2. **Noise and Redundancy:** แต่ละหน้าของเอกสารมักมีข้อความส่วนหัวและส่วนท้าย (เช่น "ราชกิจจานุเบกษา" เลขหน้า เล่ม ตอนที่) ซึ่งไม่เกี่ยวกับตัวบทกฎหมายและจะไปรบกวนโมเดล Machine Learning

บทที่ 3 การเตรียมและการจัดการข้อมูล

ส่วนนี้คือกระบวนการหลักที่ดำเนินการเสร็จสิ้นแล้ว โดยแบ่งเป็นการดึงข้อมูลดิบออกจากไฟล์ (Data Extraction) และการทำความสะอาดพร้อมจัดโครงสร้าง (Data Structuring)

3.1 การสกัดข้อความจากเอกสาร PDF (Data Extraction)

เนื่องจากรูปแบบไฟล์มีความแตกต่างกัน กระบวนการสกัดจึงถูกแบ่งเป็น 2 วิธี:

1. สำหรับข้อมูลกลุ่ม พ.ศ. 2475–2548 (Image PDF)

เนื่องจากเอกสารกลุ่มนี้เป็นไฟล์ภาพสแกนจากเอกสารรัฐธรรมนูญเก่าที่เป็นแบบพิมพ์ดีดหรือพิมพ์บนสื่ออิเล็กทรอนิกส์ แต่มีการจัดเก็บในรูปแบบรูปภาพ จึงต้องทำการสกัดตัวอักษรด้วยเทคโนโลยี Optical Character Recognition (OCR) โดยทางผู้จัดทำเลือกใช้ Typhoon OCR API (v1.5x-70b-vision-instruct) เนื่องจากเป็นโมเดลปัญญาประดิษฐ์ที่ถูกฝึกมาเพื่อรองรับโครงสร้างภาษาไทยได้ดีเยี่ยม

ตาราง 1 โค้ดสำหรับการสกัดตัวอักษรด้วยเทคนิค OCR ด้วย Typhoon OCR

```
def encode_image_to_base64(page) -> str:
    """แปลงเอกสาร PDF แต่ละหน้าให้เป็นรูปภาพ Base64 เพื่อส่งเข้า API"""
    pix = page.get_pixmap(matrix=fitz.Matrix(2.0, 2.0))
    img_bytes = pix.tobytes("jpeg")
    return base64.b64encode(img_bytes).decode("utf-8")

def ocr_constitution(meta: dict, skip_existing: bool = True):
    """ฟังก์ชันหลักในการจัดการไฟล์ภาพ PDF ดึงรูปทีละหน้าส่งเข้า Typhoon OCR"""
    cid = meta["id"]
    pdf_path = PDF_DIR / meta["filename"]

    doc = fitz.open(pdf_path)
    pages_data = []

    # วนลูปอ่านข้อมูลทุกหน้าใน PDF
    for page_num in range(doc.page_count):
        page = doc[page_num]
        base64_image = encode_image_to_base64(page)

        # โครงสร้าง Payload ที่จะส่งไปให้ Typhoon V1.5 Vision
        payload = {
            "model": "typhoon-v1.5x-70b-vision-instruct",
            "messages": [
                {"role": "user",
                 "content": [
```

```

        {"type": "text", "text": "Extract all text from this image."},
        {"type": "image_url", "image_url": {"url":
f"data:image/jpeg;base64,{base64_image}"}}
    ]}
    ]
}

# ส่ง HTTP POST Request
response = requests.post("https://api.opentypo.ai/v1/chat/completions",
headers=headers, json=payload)
extracted_text = response.json()['choices'][0]['message']['content']

pages_data.append({"page_num": page_num + 1, "text": extracted_text})

# นำ text ทุกหน้ามารวมกันเป็น full_text และบันทึกเป็น JSON
# ...

```

คำอธิบายโค้ด: ผู้จัดทำได้สร้างฟังก์ชัน `ocr_constitution` เพื่อรับค่า Metadata ของแต่ละเอกสาร โดยมีกระบวนการทำงานดังนี้:

- ใช้ไลบรารี `fitz` (PyMuPDF) เปิดไฟล์ PDF แล้วแปลงแต่ละหน้า (Page) ให้กลายเป็นรูปภาพ
- สร้างฟังก์ชัน `encode_image_to_base64` เพื่อแปลงรูปภาพให้อยู่ในฟอร์แมต Base64
- วงวน (Loop) ส่งรูปภาพ Base64 เข้าไปยัง API ของ Typhoon พร้อมคำสั่ง Prompt "Extract all text from this image."
- รวบรวมข้อความ (Text) ที่ API ตอบกลับมาในรูปแบบ JSON เก็บไว้ในตัวแปร `pages_data`

2. สำหรับข้อมูลกลุ่ม พ.ศ. 2550–2564 (Text PDF)

เอกสารกลุ่มนี้เป็น PDF ที่สร้างจากคอมพิวเตอร์และมีข้อความฝังอยู่ (Text PDF) จึงสามารถใช้ไลบรารีสกัดข้อความได้โดยตรง โดยผู้จัดทำสร้างฟังก์ชัน `_extract_text_smart` เพื่อจัดการ:

ตาราง 2 โค้ดสำหรับการสกัดตัวอักษรด้วยเทคนิคดึงข้อความดิจิทัลโดยตรง

```
def _extract_text_smart(pdf_path: Path):
    """
    ฟังก์ชันดึงข้อความจาก PDF อัจฉริยะ:
    ลองใช้ PyMuPDF ก่อน ถ้าดึงข้อความได้น้อยกว่ากำหนด จะสลับไปใช้ pdfplumber อัตโนมัติ
    """
    try:
        import fitz
        doc = fitz.open(str(pdf_path))
        # ดึงข้อความทีละหน้า
        pages = [{"page_num": i + 1, "text": page.get_text("text")} for i, page in
enumerate(doc)]
        doc.close()

        # ถ้าดึงข้อความรวมกันได้เกิน 500 ตัวอักษร ถือว่าสำเร็จ
        if sum(len(p.get("text", "")) for p in pages) > 500:
            return pages, "pymupdf"
    except Exception:
        pass

    # Fallback ไปใช้ pdfplumber ถ้าวิธีแรกไม่สำเร็จ
    import pdfplumber
    with pdfplumber.open(str(pdf_path)) as pdf:
        pages = [{"page_num": i + 1, "text": (p.extract_text() or "")} for i, p in
enumerate(pdf.pages)]
    return pages, "pdfplumber"
```

คำอธิบายโค้ด: ผู้จัดทำได้สร้างฟังก์ชัน `_extract_text_smart` เพื่อรับค่า Metadata ของแต่ละเอกสารที่เป็นรูปแบบ Text PDF โดยมีกระบวนการทำงานดังนี้:

- ใช้ไลบรารี PyMuPDF (fitz) เป็นตัวดึงข้อความหลัก เนื่องจากมีความเร็วสูง
- หากพบปัญหาในการดึงข้อความ (เช่น ข้อความได้ค่าน้อยกว่า 500 ตัวอักษร) ระบบจะสลับไปใช้ไลบรารี pdfplumber เป็นตัวสำรอง (Fallback method) อัตโนมัติ เพื่อรักษาความสมบูรณ์ของข้อมูลให้ได้มากที่สุด

3.2 การจัดการและทำความสะอาดข้อมูลก่อนนำไปใช้ (Cleaning & Structuring)

ในการดำเนินการพัฒนาโครงการได้มีการใช้งานเครื่องมือฮาร์ดแวร์ต่าง ๆ ดังตารางที่ 2 ข้อมูลที่สกัดออกมาทั้งจาก OCR และ Text Extraction ถูกนำเข้ากระบวนการทำความสะอาดข้อมูล (Data Cleaning Pipeline) เพื่อลดสัญญาณรบกวน (Noise) และแก้ไขข้อผิดพลาดทางภาษา ผ่าน 4 ฟังก์ชันย่อย และรวบรวมด้วยฟังก์ชันหลัก `extract_constitution` ดังนี้:

1. การแปลงอักขระ (Normalization)

ผู้จัดทำใช้ฟังก์ชัน `_fix_pua` เพื่อแมป (Map) รหัสอักขระเก่าที่อยู่ในช่วง F700-F71A ให้กลับมาเป็น Unicode ตามมาตรฐาน และฟังก์ชัน `_normalize` ร่วมกับไลบรารี `unicodedata` เพื่อปรับอักขระให้เป็น NFC Form และลบอักขระควบคุม (Control Characters) ที่ซ่อนอยู่

2. การซ่อมแซมคำผิดด้วย Regular Expression

เนื่องจากในเอกสาร Text PDF บางฉบับ มีการเข้ารหัสตัว "สระอำ" ผิดพลาด โดยถูกตัดแบ่งเป็นเว้นวรรคและสระอา (เช่น คำว่า "จ านวน") ผู้จัดทำจึงสร้างฟังก์ชัน `_fix_sara_am` เพื่อใช้ RegEx ตรวจสอบพยัญชนะที่ตามด้วยเว้นวรรคและสระอา แล้วแทนที่ให้กลับเป็นสระอำ (า) แบบอัตโนมัติ

3. การกำจัดส่วนเกินของราชกิจจานุเบกษา

เอกสารแต่ละหน้ามักมีข้อความส่วนหัวและเลขหน้าของราชกิจจานุเบกษาปรากฏอยู่ ซึ่งไม่เกี่ยวกับเนื้อหารัฐธรรมนูญ ผู้จัดทำจึงสร้างฟังก์ชัน `_remove_headers` ที่ใช้ RegEx กวาดหาแพทเทิร์นดังกล่าวและลบทิ้ง

ตาราง 3 โค้ดสำหรับ Data Cleaning Pipeline

```
import re
import unicodedata

def _fix_pua(text: str) -> str:
    """แปลงรหัสอักขระเก่า (PUA) จากฟอนต์เช่น AngsanaUPC ให้กลับเป็น Unicode ปกติ"""
    if not text: return ""
    pua_map = {
        0xF700: 0x0E31, 0xF701: 0x0E34, 0xF702: 0x0E35, 0xF703: 0x0E36, 0xF704: 0x0E37,
        0xF705: 0x0E48, 0xF706: 0x0E49, 0xF707: 0x0E4A, 0xF708: 0x0E4B, 0xF709: 0x0E4C,
        # ... (แมปช่วง F700-F71A ไปหา 0E31-0E4C) ...
    }
    return text.translate(pua_map)

def _normalize(text: str) -> str:
    """จัดการทำ Normalization รูปแบบ NFC และลบช่องว่างส่วนเกิน"""
    if not text: return ""
    text = unicodedata.normalize("NFC", text)
    text = re.sub(r"[\x00-\x08\x0b\x0c\x0e-\x1f]", "", text) # ลบตัวควบคุม
    text = re.sub(r"[\t]+", " ", text) # ยุบช่องว่างให้เหลือ 1 เคาะ
    return text.strip()
```

```

def _fix_sara_am(text: str) -> str:
    """แก้ปัญหาสระอำแยกส่วน (เช่น 'จ ำนวน' -> 'จำนวน')"""
    SARA_AM_PATTERN = re.compile('([ก-ฮ][^:~?]) (า)')
    return SARA_AM_PATTERN.sub(lambda m: m.group(1) + 'า', text)

def _remove_headers(text: str) -> str:
    """ตัดข้อความส่วนหัว/ส่วนท้ายของเอกสารราชกิจจานุเบกษาออก"""
    patterns = [
        r"\u0E2B\u0E19\u0E49\u0E32\s*\d+\s*\u0E40\u0E25\u0E48\u0E21\s*\d+.*?\u0E23\u0E32\u0E0A\u0E01\u0E34\u0E08\u0E08\u0E32\u0E19\u0E38\u0E40\u0E1A\u0E01\u0E29\u0E32[^\n]*",
        r"^\s*-\s*\d+\s*-\s*$",
        r"^\s*\d+\s*$",
    ]
    for pattern in patterns:
        text = re.sub(pattern, "", text, flags=re.MULTILINE | re.IGNORECASE)
    return text

```

4. การบันทึกข้อมูลแบบมีโครงสร้าง (JSON Structuring)

กระบวนการทำงานทั้งหมดจะถูกประมวลผลผ่านฟังก์ชันรวม `extract_constitution` เพื่อทำการนำข้อมูลแต่ละหน้า (Page) มาวนลูปทำความสะอาดทีละฟังก์ชัน แล้วรวบรวมเก็บเป็น Dictionary เดียว และบันทึกผลลัพธ์เป็นไฟล์ .json ตามโครงสร้างที่ออกแบบไว้ในบทที่ 2.3 ทำให้ได้ไฟล์ที่มีโครงสร้างมาตรฐานเดียวกัน พร้อมส่งต่อให้ขั้นตอนการสร้างแบบจำลอง (Modeling) ทันที

ตาราง 4 โค้ดสำหรับ Data Pipeline Structuring

```
def extract_constitution(meta: dict, skip_existing: bool = True):
    """ฟังก์ชันไปป์ไลน์หลัก รวบรวมการสกัดข้อความ การทำความสะอาด และจัดโครงสร้าง"""
    pdf_path = PDF_DIR / meta["filename"]

    # 1. ดึงข้อความจากไฟล์
    pages_data, method = _extract_text_smart(pdf_path)

    # 2. นำข้อความแต่ละหน้าผ่านกระบวนการทำความสะอาด (Cleaning Pipeline)
    for p in pages_data:
        text = _fix_pua(p.get("text", ""))
        text = _normalize(text)
        text = _fix_sara_am(text)
        text = _remove_headers(text)
        p["text"] = text

    # 3. นำข้อความทุกหน้ามาต่อรวมกัน
    full_text = _normalize("\n\n".join(p["text"] for p in pages_data if p.get("text")))

    # 4. จัดโครงสร้างข้อมูล Metadata ให้อยู่ในฟอร์แมต JSON Schema
    result = {
        "id": meta["id"],
        "year_th": meta["year_th"],
        "name_short": meta["name_short"],
        "source_type": "text_pdf",
        "processing_method": method,
        "total_pages": len(pages_data),
        "pages": pages_data,
        "full_text": full_text,
        "metadata": {
            "total_chars": len(full_text),
            "total_words_approx": len(full_text.split()),
        },
    }
    return result
```

3.3 การจัดโครงสร้างข้อมูลเชิงลึก (Deep Structure Parsing)

หลังจากผ่านกระบวนการ Extraction และ Clean ข้อมูลใน 3.1 และ 3.2 มาแล้ว ข้อมูลที่ได้ยังอยู่ในรูปแบบ raw text ต่อหน้า ซึ่งยังไม่สามารถนำไปวิเคราะห์เชิงโครงสร้างได้โดยตรง คณะผู้จัดทำจึงพัฒนา Post-Processor สำหรับแปลงข้อมูลให้มีโครงสร้างแบบ Hierarchical ตาม 3 ระดับของรัฐธรรมนูญไทย ได้แก่ หมวด (Chapter), ส่วนที่ (Part), และ มาตรา (Section) แล้ว export ออกมาในรูปแบบ JSON และ CSV

1. การ parse โครงสร้างหมวด ส่วนที่ และมาตรา

ฟังก์ชัน `_split_by_boundary` มีหน้าที่ตรวจจับขอบเขตของแต่ละหมวดด้วย Regular Expression โดยรองรับทั้งรูปแบบ หมวดที่ N, บทที่ N และหมวดพิเศษ เช่น บททั่วไป และ บทเฉพาะกาล จากนั้นฟังก์ชัน `_parse_parts_from_segment` ใช้สำหรับตรวจว่าในหมวดนั้นมี ส่วนที่ ซ่อนอยู่ด้วยหรือไม่ และฟังก์ชัน `_parse_sections_from_segment` ใช้การ slice ตาม position ของคำว่า มาตรา N แทนการใช้ Regex แบบ non-greedy เพื่อให้ข้อมูลในแต่ละมาตราครบถ้วนไม่ถูกตัดกลางคัน

ตาราง 5 ฟังก์ชัน `_parse_sections_from_segment` และ `_merge_cross_chapter_duplicates`

```
def _parse_sections_from_segment(text: str) -> List[Any]:
    """ ค้นหาและแยกเนื้อหาของแต่ละมาตราโดยใช้ตำแหน่งเริ่มต้นของ มาตรา N ใช้วิธีตัดข้อความ (Slice) ตาม
    ตำแหน่งที่พบเพื่อให้อ่านข้อมูลได้ครบถ้วนและแม่นยำกว่าการใช้ Regex แบบปกติ """
    matches = list(_RE_SECTION_START.finditer(text))
    raw_sections: List[Any] = []

    for i, m in enumerate(matches):
        sec_num = int(m.group(1))
        content_start = m.end()
        content_end = matches[i + 1].start() if i + 1 < len(matches) else len(text)

        # ทำความสะอาดข้อความโดยการยุบช่องว่างที่ซ่อนให้เหลือเพียงช่องเดียว
        sec_text = re.sub(r'\s+', ' ', text[content_start:content_end]).strip()
        if sec_text:
            raw_sections.append(Section(section_number=sec_num, text=sec_text))

    # รวมมาตราที่มีเลขซ้ำกัน (จัดการกรณีที่ได้ผลจาก OCR มีหัวข้อแทรกกลางเนื้อหามาตรา)
    merged: List[Any] = []
    for sec in raw_sections:
        if merged and merged[-1].section_number == sec.section_number:
            # รวมเนื้อหาเข้ากับมาตราก่อนหน้า
            merged[-1] = Section(
                section_number=sec.section_number,
                text=merged[-1].text + ' ' + sec.text
            )
        else:
            merged.append(sec)
```

```

return merged

def _merge_cross_chapter_duplicates(chapters: List[Any]) -> List[Any]:
    """ รวมเนื้อหาของมาตราที่มีเลขซ้ำกันข้ามบท (Chapter) โดยข้อมูลซ้ำจะถูกนำไปต่อท้ายเนื้อหาในบทแรกที่พบ
    มาตราเลขนั้นๆ """
    seen: Dict[int, int] = {} # เก็บค่า section_number -> index ของบทที่พบครั้งแรก

    for ci, ch in enumerate(chapters):
        new_secs = []
        for sec in ch.sections:
            n = sec.section_number
            if n in seen:
                # หากเคยพบมาตรานี้แล้วในบทก่อนหน้านี้ ให้นำเนื้อหาไปรวมกัน
                orig_chapter_index = seen[n]
                orig_secs = chapters[orig_chapter_index].sections
                for si, s in enumerate(orig_secs):
                    if s.section_number == n:
                        orig_secs[si] = Section(
                            section_number=n,
                            text=s.text + ' ' + sec.text
                        )
                        break
            else:
                # หากยังไม่เคยพบ ให้บันทึก index ของบทปัจจุบันไว้
                seen[n] = ci
                new_secs.append(sec)

        # อัปเดตรายการมาตราในบทปัจจุบัน (ตัดมาตราที่ถูกย้ายไปรวมที่อื่นออก)
        ch.sections = new_secs
    return chapters

```

2. การตรวจจับประเภทฉบับ (Original / Amendment)

รัฐธรรมนูญบางฉบับที่มีการประกาศออกมา เป็นฉบับแก้ไขเพิ่มเติม ไม่ใช่ฉบับประกาศใหม่ ดังนั้น ฟังก์ชัน `_detect_constitution_type` ไว้ตรวจสอบ keyword เช่น แก้ไขเพิ่มเติม หรือ amendment ใน `name_short` และเนื้อหาต้น แล้วส่งคืนค่า `constitution_type` และ `amends_year` เพื่อให้การวิเคราะห์เปรียบเทียบสามารถ reference กลับไปยังฉบับหลักได้ถูกต้อง

ตาราง 6 ฟังก์ชัน `_detect_constitution_type`

```
# คำสำคัญสำหรับตรวจสอบว่าเป็นฉบับแก้ไขเพิ่มเติมหรือไม่
_AMENDMENT_KEYWORDS = [
    'แก้ไขเพิ่มเติม', 'ฉบับแก้ไข', 'แก้ไข พ.ศ.', 'amendment',
]

def _detect_constitution_type(raw_data: Dict) -> Tuple[str, Optional[int]]:
    """ ตรวจสอบประเภทของรัฐธรรมนูญว่าเป็นฉบับหลัก (Original) หรือฉบับแก้ไขเพิ่มเติม (Amendment)
    พร้อมทั้งระบุปีที่ได้รับการแก้ไขหากเป็นฉบับแก้ไขเพิ่มเติม """
    # รวมชื่อย่อและเนื้อหา 500 ตัวอักษรแรกเพื่อใช้ในการค้นหาคำสำคัญ
    name = raw_data.get('name_short', '') + ' ' + raw_data.get('full_text', '')[:500]

    # ตรวจสอบว่ามีคำสำคัญที่ระบุว่าเป็นฉบับแก้ไขเพิ่มเติมหรือไม่
    is_amendment = any(kw in name for kw in _AMENDMENT_KEYWORDS)

    if is_amendment:
        # ค้นหาปี พ.ศ. ที่ระบุในบริบทของการแก้ไขเพิ่มเติม
        year_match = re.search(r'แก้ไข.*?(\d{4})', name)
        amends_year = int(year_match.group(1)) if year_match else None
        return 'amendment', amends_year
    return 'original', None
```

3. การ Export JSON และ CSV

ฟังก์ชัน `_iter_all_sections` เป็น Generator ที่ traverse โครงสร้าง chapters ไป parts ไป sections แล้ว yield dict row ออกมาทีละ 1 มาตรา ซึ่งนำไปใช้ทั้งใน การบันทึก JSON แบบ nested และ CSV แบบ flat ทำให้ code ไม่ซ้ำซ้อน

ตาราง 7 ฟังก์ชัน `_iter_all_sections` สำหรับ Export

```
def _iter_all_sections(data: Dict[str, Any]) -> Generator[Dict[str, Any], None, None]:
    """ Generator สำหรับวนลูปข้อมูลมาตราทั้งหมด โดยคืนค่าเป็น Dictionary หนึ่งแถวต่อหนึ่งมาตรา
    รองรับโครงสร้างข้อมูลแบบลำดับชั้น: หมวด (Chapter) -> ส่วน (Part) -> มาตรา (Section) """
    # กำหนดข้อมูลพื้นฐานที่ใช้ร่วมกันในทุกมาตรา
    base = {
        'constitution_id': data['id'],
```

```

        'year_th': data['year_th'],
        'year_ce': data['year_ce'],
        'constitution_type': data.get('constitution_type'),
        'amends_year': data.get('amends_year', ''),
        'era': data.get('era', ''),
        'regime_type': data.get('regime_type', ''),
    }

for chap in data['chapters']:
    # สร้าง Context ของหมวด (Chapter)
    chap_ctx = {
        **base,
        'chapter_number': chap['chapter_number'],
        'chapter_title': chap['chapter_title']
    }

    # วนลูปมาตราที่สังกัดอยู่ในหมวดโดยตรง (กรณีที่ไม่มีการแบ่งเป็น "ส่วนที่")
    for sec in chap.get('sections', []):
        yield {
            **chap_ctx,
            'part_number': '',
            'part_title': '',
            'section_number': sec['section_number'],
            'section_text': sec['text']
        }

    # วนลูปมาตราที่แบ่งตาม "ส่วนที่" (Part) ภายในหมวดนั้นๆ
    for part in chap.get('parts', []):
        for sec in part.get('sections', []):
            yield {
                **chap_ctx,
                'part_number': part['part_number'],
                'part_title': part['part_title'],
                'section_number': sec['section_number'],
                'section_text': sec['text']
            }

```


3.4 สรุปผลลัพธ์การเตรียมข้อมูล (Data Preparation Results)

หลังจากผ่านกระบวนการ Extraction Cleaning และจัดโครงสร้างข้อมูลทั้งหมดแล้ว จะได้ผลลัพธ์เป็นไฟล์ 2 รูปแบบต่อ 1 ฉบับ ได้แก่ Structured JSON และ Flat CSV รวมทั้งสิ้น 38 ฉบับ ครอบคลุมรัฐธรรมนูญและธรรมนูญการปกครองไทยทุกฉบับ

1. โครงสร้าง Structured JSON

ไฟล์ JSON มีโครงสร้างแบบ Hierarchical 3 ระดับ ได้แก่ ระดับ Root เก็บ metadata ของฉบับ ระดับ chapters เก็บข้อมูลรายหมวด และระดับ sections/parts เก็บรายมาตรา ซึ่งจะมีรูปแบบการจัดเก็บเป็นแบบไฟล์ structured_YYYY.json โดยมี field สำคัญดังตารางที่ 8

ตาราง 8 โครงสร้าง field ใน Structured JSON

Field	Type	ตัวอย่าง	คำอธิบาย	Field
id	string	const_2475	unique ID ของฉบับ	id
year_th / year_ce	int	2475 / 1932	ปี พ.ศ. และ ค.ศ.	year_th / year_ce
constitution_type	string	original / amendment	ประเภทฉบับ	constitution_type
amends_year	int null	2475 / null	ปีฉบับที่ถูกแก้ไข	amends_year
era / regime_type	string	early_democracy	ยุคสมัยและระบอบ	era / regime_type
preamble	string	สมเด็จพระ...	คำปรารภ (สูงสุด 2,000 chars)	preamble
summary.total_chapters	int	7	จำนวนหมวดทั้งหมด	summary.total_chapters

2. โครงสร้าง Flat CSV

ไฟล์ CSV มีรูปแบบ 1 row ต่อ 1 มาตรา เรียงตาม section_number เหมาะสำหรับการนำไปวิเคราะห์ด้วย ซึ่งเนื้อหาทั้งหมดจะถูกจัดเก็บในรูปแบบไฟล์ sections_YYYY.csv จากนั้นจะมีไฟล์ all_sections_combined.csv จะรวมทุกฉบับไว้ด้วยกัน สำหรับ cross-version analysis

ตาราง 9 โครงสร้าง column ใน Flat CSV

Column	Type	ตัวอย่าง	คำอธิบาย
constitution_id	string	const_2475	unique ID ของฉบับ
year_th / year_ce	int	2475 / 1932	ปี พ.ศ. และ ค.ศ.
constitution_type	string	original	original หรือ amendment
amends_year	int empty	2475	ปีฉบับที่แก้ไข (ถ้าเป็น amendment)
era / regime_type	string	early_democracy	ยุคสมัยและระบอบ
chapter_number	int	1	เลขหมวด
chapter_title	string	พระมหากษัตริย์	ชื่อหมวด
part_number	int empty	2	เลขส่วน (ว่างถ้าไม่มี ส่วนที่)
part_title	string empty	ส่วนที่ 2 สภาผู้แทน	ชื่อส่วน
section_number	int	3	เลขมาตรา
section_text	string	องค์พระมหากษัตริย์...	เนื้อหามาตรา (cleaned)

3. ผลลัพธ์ภาพรวม

ตารางที่ 10 แสดงสรุปจำนวนหมวด ส่วน และมาตราของรัฐธรรมนูญไทยฉบับหลักที่ผ่านกระบวนการ Post-Processing เรียบร้อยแล้ว

ตาราง 10 สรุปผลการจัดโครงสร้างข้อมูลรัฐธรรมนูญฉบับหลัก

ปี พ.ศ.	เอกสาร	หมวด	ส่วน	มาตรา	จำนวนคำ (โดยประมาณ)
2475	Constitution 2475	7	0	68	987
2489	Constitution 2489	9	0	96	2,021
2492	Constitution 2492	11	6	223	3,238
2517	Constitution 2517	12	4	238	4,033
2540	Constitution 2540	15	8	336	7,693
2550	Constitution 2550	15	7	309	9,367
2560	Constitution 2560	16	10	279	7,939

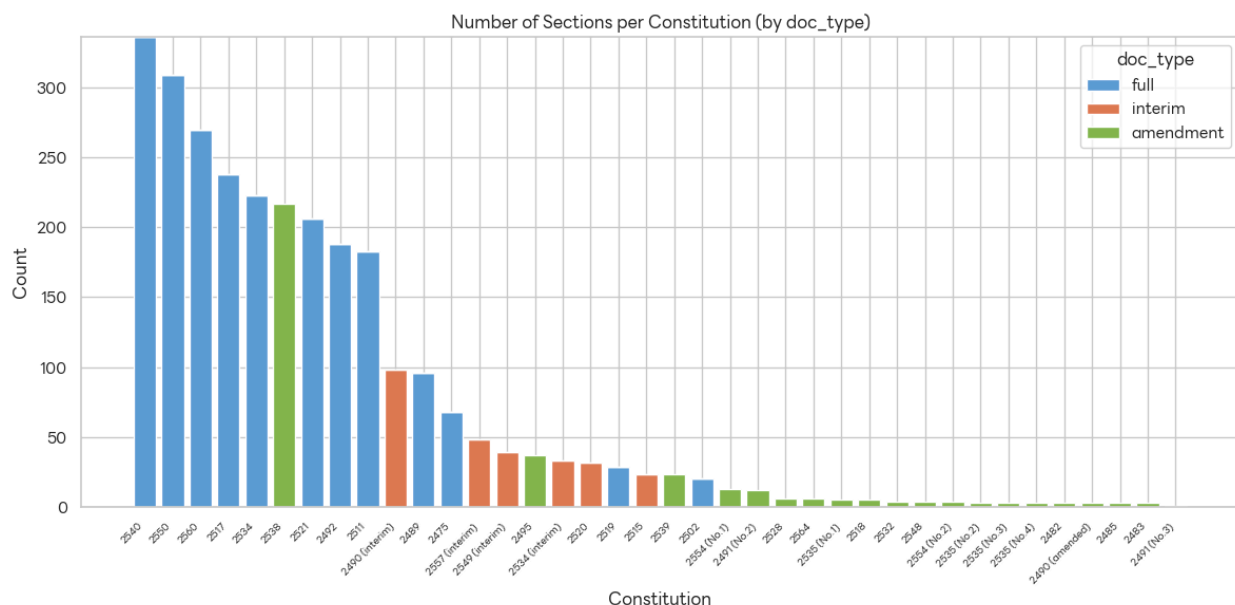
(รวมทุกฉบับ 38 เอกสาร ได้มาตราทั้งหมดประมาณ 3,200 มาตรา รวมประมาณ 61,301 คำ พร้อม export เป็น all_sections_combined.csv สำหรับ cross-version analysis)

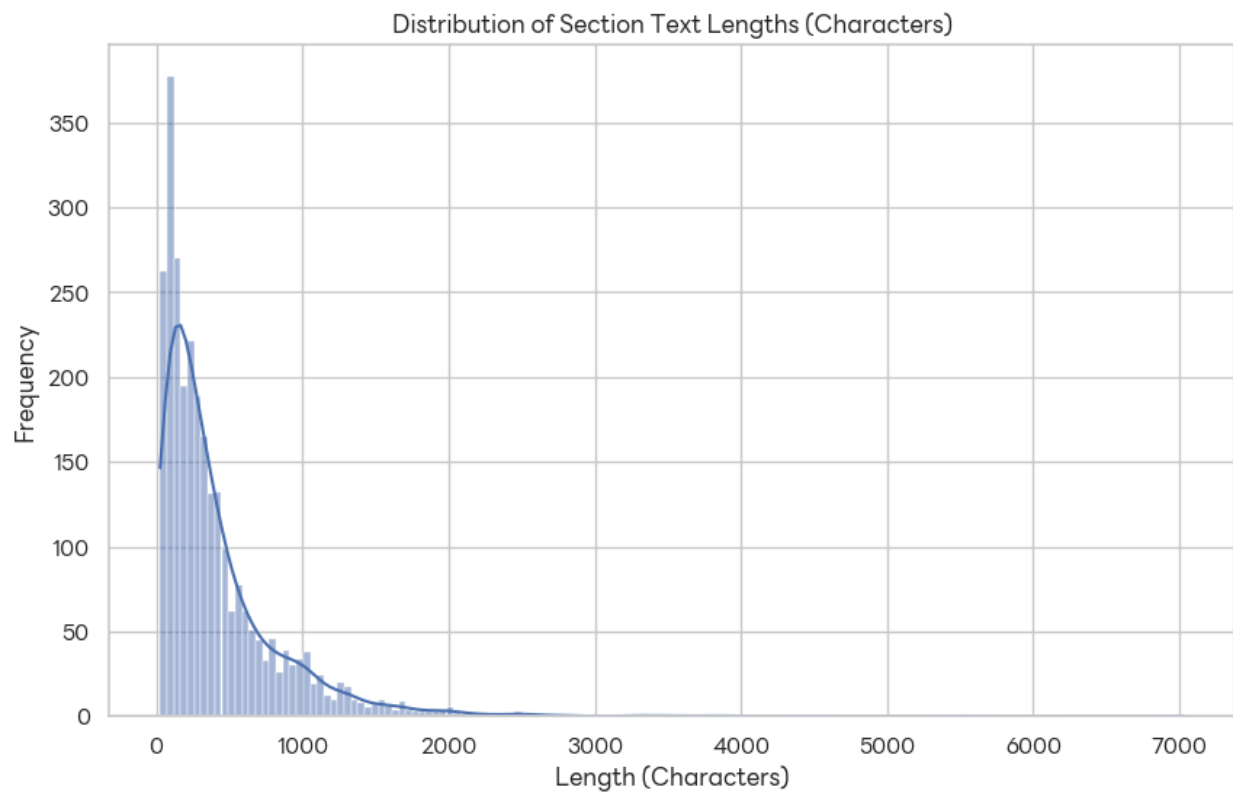
บทที่ 4 การวิเคราะห์ข้อมูล (Exploratory Data Analysis & Visualization)

หลังจากผ่านกระบวนการเตรียมข้อมูลแล้ว ผู้จัดทำได้นำชุดข้อมูล ซึ่งประกอบด้วย 2,797 แถว 21 คอลัมน์ ครอบคลุมรัฐธรรมนูญไทย 38 ฉบับ ตั้งแต่ พ.ศ. 2475–2564 มาวิเคราะห์เชิงสำรวจ (EDA) เพื่อทำความเข้าใจโครงสร้าง พฤติกรรมทางภาษา และพัฒนาการของรัฐธรรมนูญตามกาลเวลา

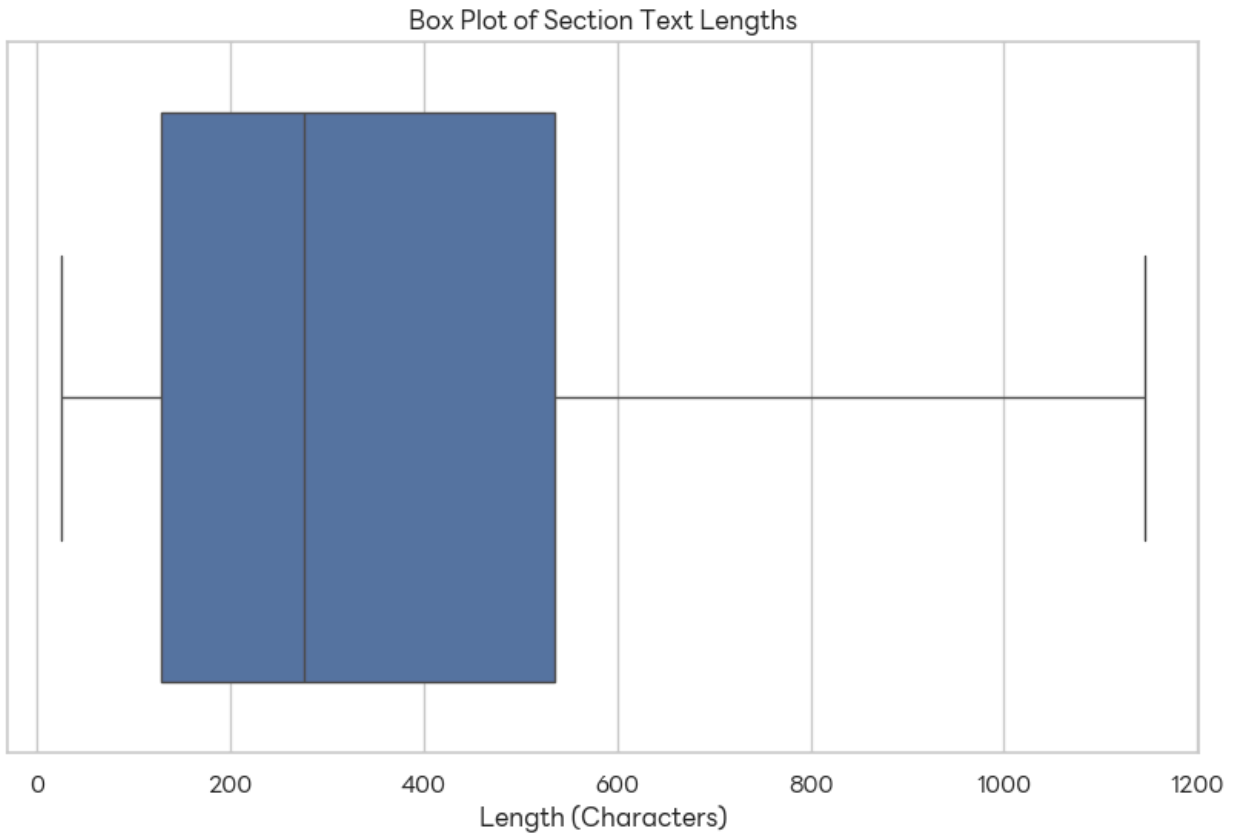
4.1. โครงสร้างของคลังข้อมูล (Corpus Structure)

การวิเคราะห์เบื้องต้นเริ่มจากการพิจารณาจำนวนมาตราต่อรัฐธรรมนูญแต่ละฉบับ โดยแสดงผลด้วยแผนภูมิแท่งแบบ Stacked ที่แบ่งสีตามประเภทเอกสาร `doc_type` ได้แก่ รัฐธรรมนูญฉบับสมบูรณ์ (full) รัฐธรรมนูญชั่วคราว (interim) และฉบับแก้ไขเพิ่มเติม (amendment) ผลที่ได้พบว่ารัฐธรรมนูญ พ.ศ. 2540 และ 2550 มีจำนวนมาตราสูงสุด สะท้อนความพยายามในการออกแบบระบบการเมืองที่ครอบคลุมในยุคประชาธิปไตย ขณะที่รัฐธรรมนูญชั่วคราวมักมีจำนวนมตราน้อยกว่าอย่างเห็นได้ชัด





รูปที่ 4.2. Histogram และ KDE แสดงการกระจายตัวของจำนวนตัวอักษร ในแต่ละมาตรา

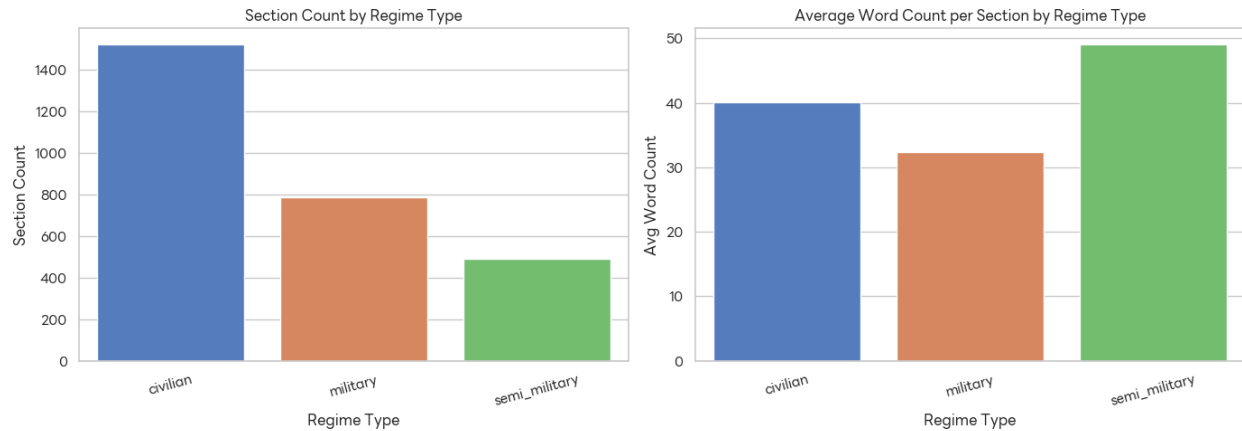


รูปที่ 4.3. Box Plot แสดงการกระจายตัวของความยาวข้อความแยกตามรัฐธรรมนูญแต่ละฉบับ

4.2. บริบทเชิงการเมืองของข้อมูล (Political Context)

เพื่อเชื่อมโยงข้อมูลเข้ากับบริบทการเมืองไทย ผู้จัดทำได้วิเคราะห์ตามตัวแปร `regime_type` และ `era` ซึ่งระบุประเภทรัฐบาลและยุคสมัยของรัฐธรรมนูญแต่ละฉบับ พบว่า

- รัฐธรรมนูญในยุครัฐบาลพลเรือน (civilian) มีจำนวนมาตราเฉลี่ยสูงสุด และมีค่าเฉลี่ยจำนวนคำต่อมาตราสูงกว่ากลุ่มอื่น แสดงถึงความละเอียดในการบัญญัติกฎหมาย
- รัฐธรรมนูญในยุครัฐบาลกึ่งทหาร (semi_military) มีความยาวมาตราโดยเฉลี่ยต่ำกว่า สอดคล้องกับลักษณะการบัญญัติที่กระชับและจำกัดขอบเขต
- การเปรียบเทียบ Box Plot ระหว่าง doc_type ใน 8 ยุคสมัย (era) ยืนยันว่ารัฐธรรมนูญชั่วคราวในทุกยุคมีความยาวมตราน้อยและกระจายตัวแคบกว่ารัฐธรรมนูญฉบับสมบูรณ์อย่างสม่ำเสมอ

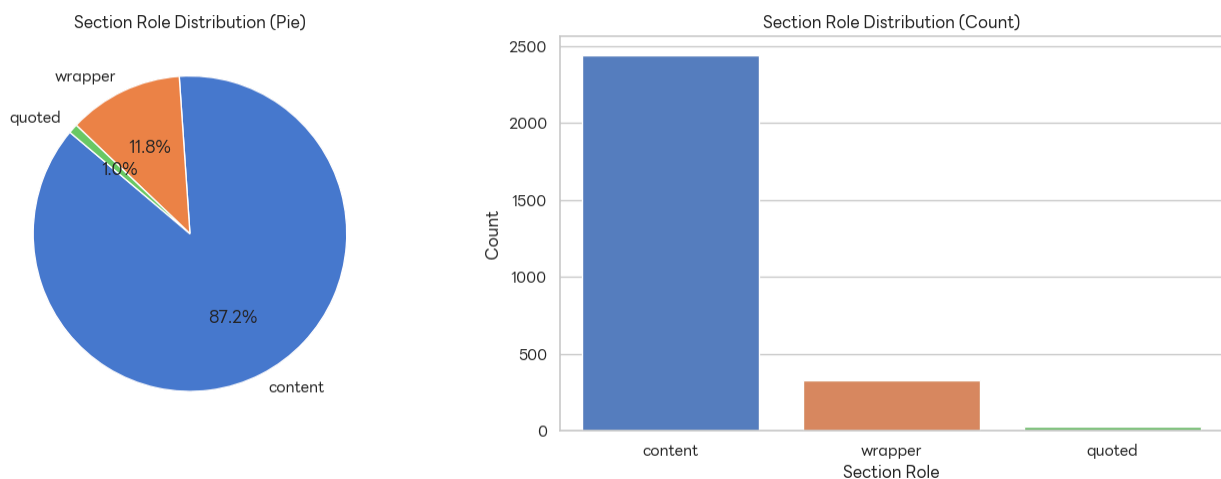


รูปที่ 4.2.1 แผนภูมิแท่งแสดงจำนวนมาตราและค่าเฉลี่ยจำนวนคำต่อมาตรา แยกตามประเภทระบอบการปกครอง

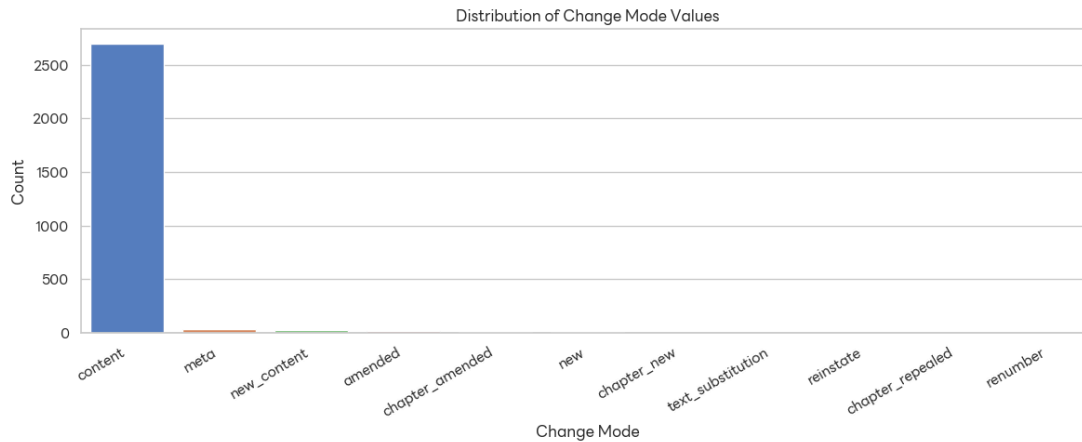
4.3. โครงสร้างบทบาทของมาตรา (Section Role & Change Mode)

ตัวแปร **section_role** และ **change_mode** ช่วยเปิดเผยโครงสร้างภายในของรัฐธรรมนูญในมิติที่ไม่เคยวิเคราะห์มาก่อน

- จากแผนภูมิวงกลมและแท่งของ **section_role** พบว่ามาตราประเภท **content** (มาตราเนื้อหาหลัก) ครอบคลุมสัดส่วนสูงสุด รองลงมาคือ **wrapper** (มาตราบทนำ/บทเฉพาะกาล) และ **header** ตามลำดับ สัดส่วนดังกล่าวสะท้อนว่ารัฐธรรมนูญไทยมีโครงสร้างเชิงกฎหมายที่เป็นระบบ
- จากแผนภูมิแท่งของ **change_mode** พบว่ารูปแบบ **content** (มาตราต้นฉบับไม่มีการแก้ไข) มีจำนวนมากที่สุด แต่ก็ยังมีมาตราประเภท **amended** และ **repealed** จำนวนมากพอสมควร บ่งชี้ว่ารัฐธรรมนูญไทยมีวัฒนธรรมการแก้ไขบทบัญญัติบ่อยครั้งตลอดประวัติศาสตร์



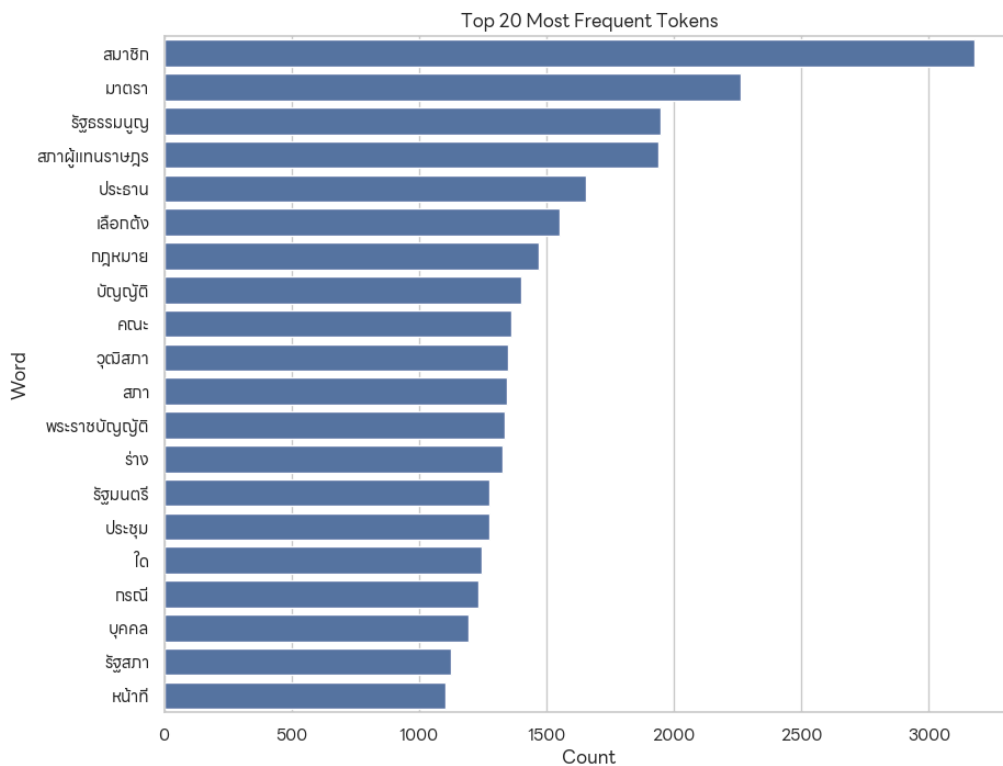
รูปที่ 4.3.1 แผนภูมิวงกลมและแท่งแสดงการกระจายตัวของบทบาทมาตรา (section_role)



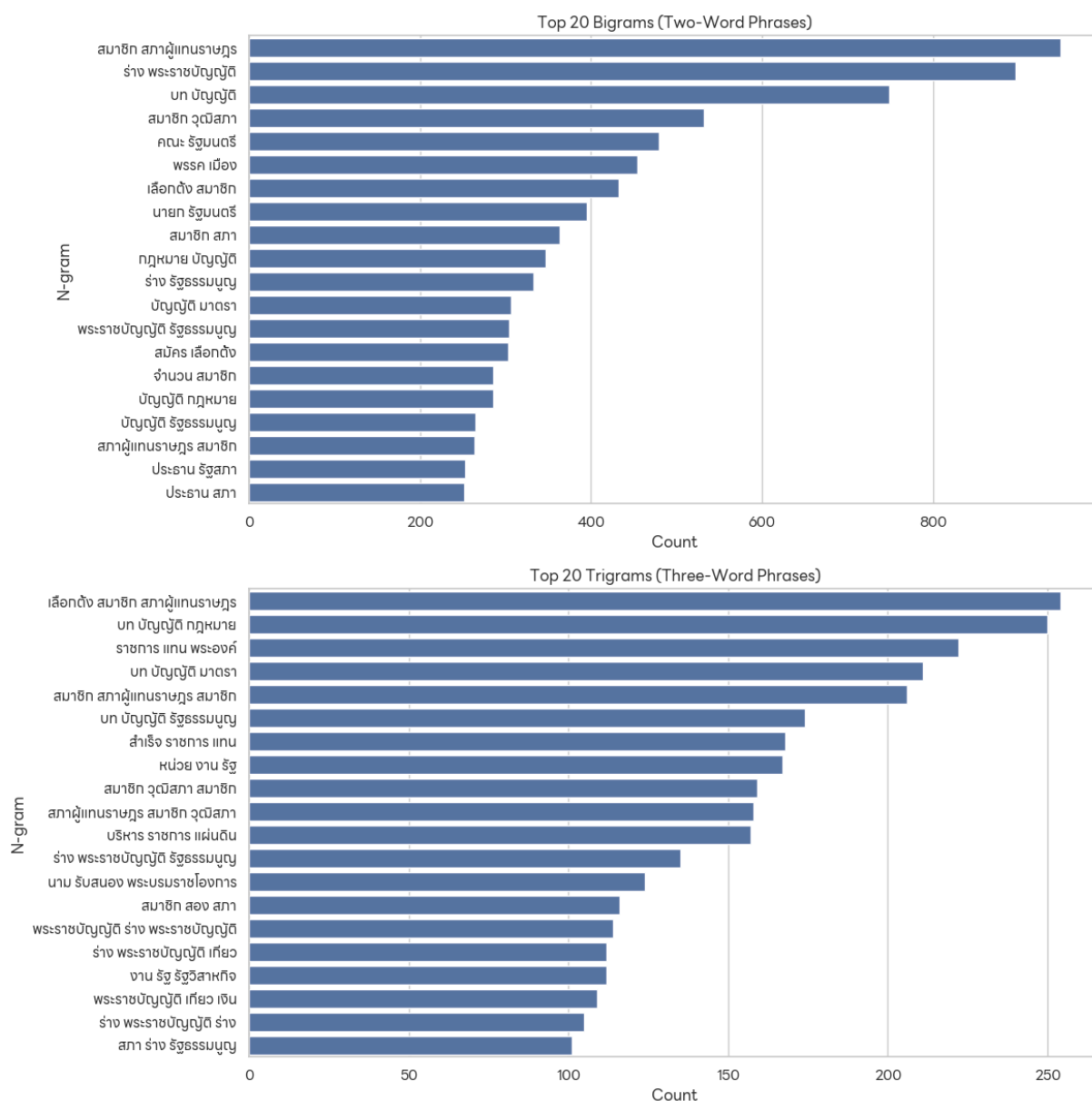
รูปที่ 4.3.2 แผนภูมิแท่งแสดงการกระจายตัวของรูปแบบการเปลี่ยนแปลงมาตรา (change_mode)

4.4 การวิเคราะห์คลังคำและเนื้อหา (Lexical & Content Analysis)

Bag-of-Words และ N-Gram: คำที่ปรากฏบ่อยที่สุดในคลังข้อมูลได้แก่ คำว่า "รัฐธรรมนูญ" "มาตรา" "พระมหากษัตริย์" และ "รัฐสภา" ซึ่งเป็นแกนกลางของเนื้อหาที่คาดไว้ ขณะที่การวิเคราะห์ Bigram และ Trigram เผยให้เห็นวลีสำคัญเชิงกฎหมาย เช่น "สมาชิกสภาผู้แทนราษฎร"



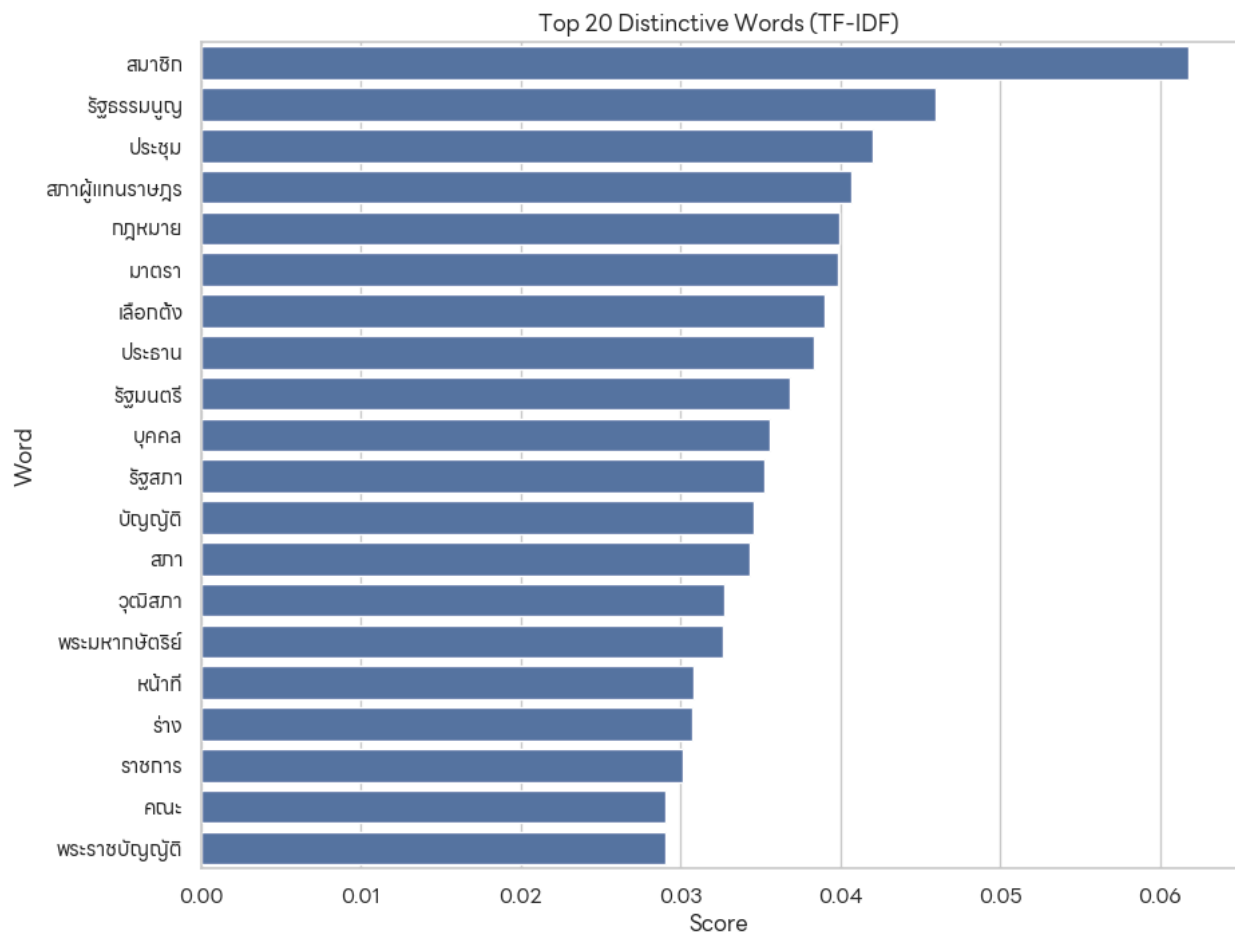
รูปที่ 4.4.1 แผนภูมิแท่งแสดง 20 คำที่ปรากฏบ่อยที่สุดในคลังข้อมูลทั้งหมด



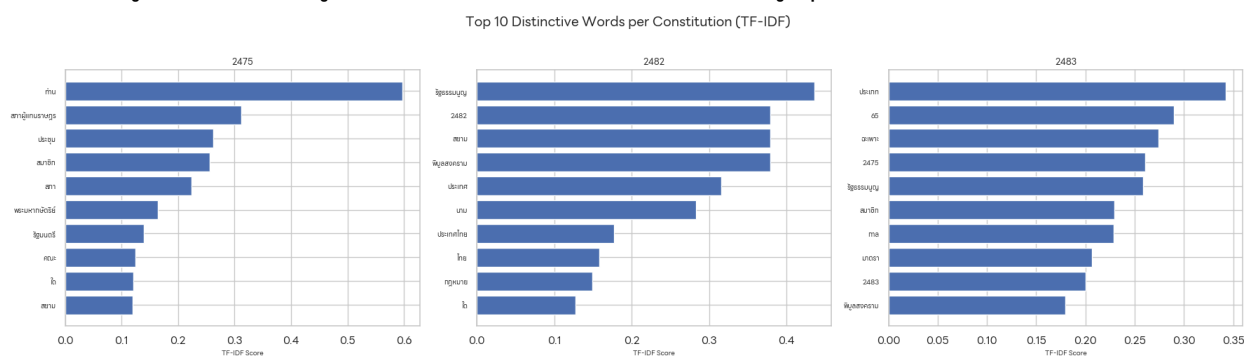
รูปที่ 4.4.2 แผนภูมิแท่งแสดง Bigram และ Trigram ที่พบบ่อยที่สุด 20 อันดับแรก

TF-IDF: การวิเคราะห์ TF-IDF ระดับ Corpus ช่วยกรองคำที่มีความโดดเด่นเฉพาะตัวออกจากคำที่พบทั่วไป ผู้จัดทำได้สร้าง Interactive Slider Widget ให้สามารถสำรวจ n-gram ที่มีค่า TF-IDF สูงสุดในระดับ 1 ถึง 10 คำ และ Interactive Dropdown สำหรับเปรียบเทียบคำเด่นของรัฐธรรมนูญแต่ละฉบับ ซึ่งพบว่ารัฐธรรมนูญแต่ละยุคมีคำศัพท์เฉพาะที่สะท้อนบริบทการเมืองในช่วงเวลานั้น

$$w_{i,j} = tf_{i,j} \times \log\left(\frac{N}{df_i}\right)$$



รูปที่ 4.4.3 แผนภูมิแท่งแสดง 20 คำที่มีค่าเฉลี่ย TF-IDF สูงสุดในระดับ Corpus ทั้งหมด

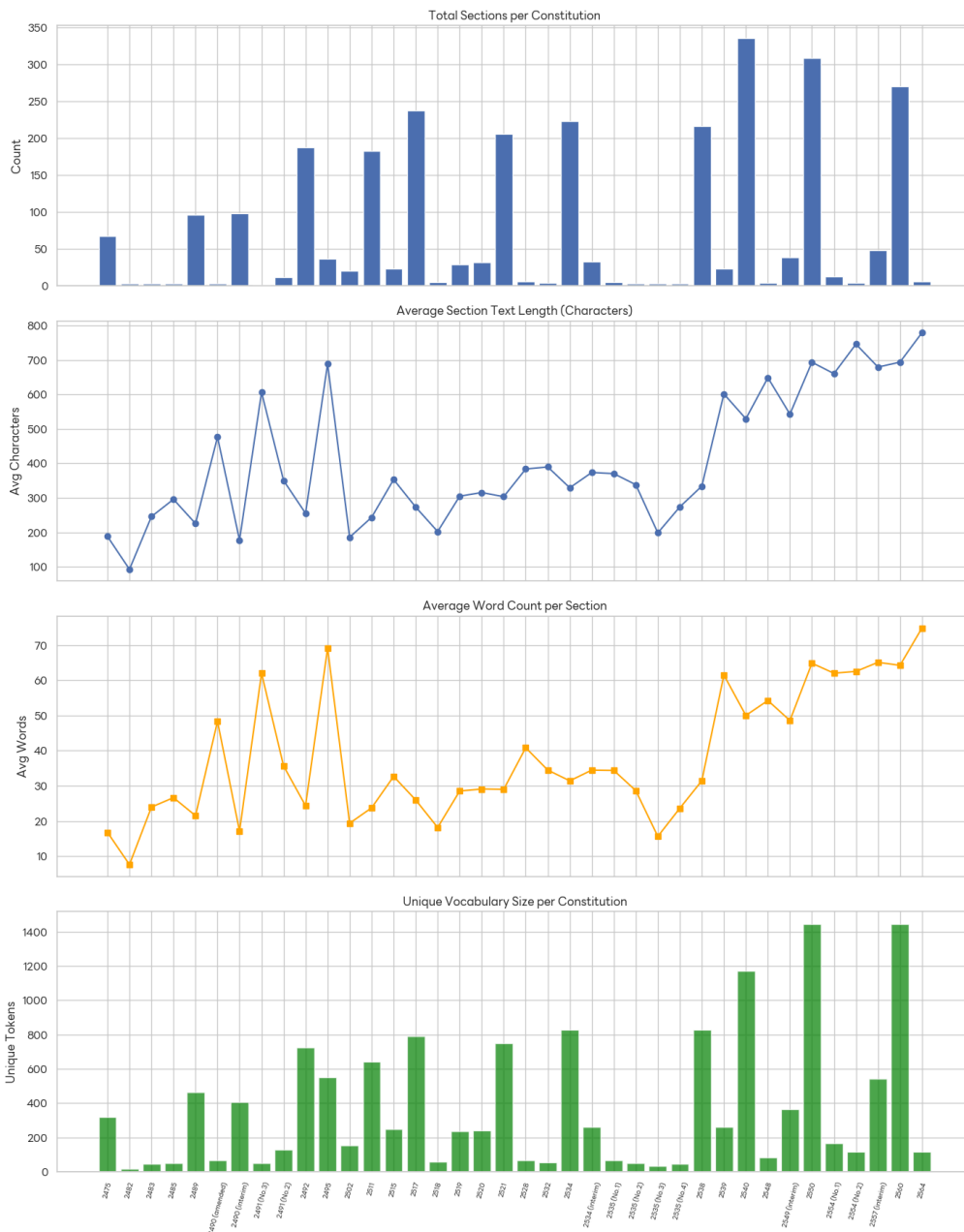


รูปที่ 4.4.4 Grid แผนภูมิแท่งแสดง 10 คำเด่นของรัฐธรรมนูญ 3 ฉบับแรกตามลำดับปีพระพุทธศักราช

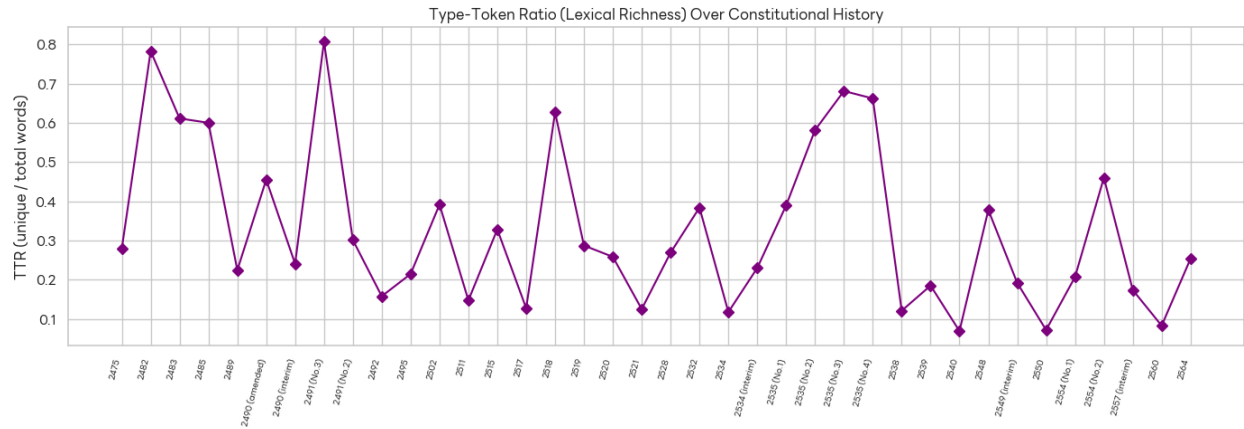
4.5 พัฒนาการของคลังคำตามกาลเวลา (Vocabulary Evolution)

กราฟการเติบโตของคลังคำสะสม (Cumulative Vocabulary Growth) แสดงให้เห็นว่าคำศัพท์ในรัฐธรรมนูญไทยขยายตัวอย่างต่อเนื่องตลอด 89 ปี โดยมีจุดพุ่งขึ้นชัดเจนในช่วงรัฐธรรมนูญ พ.ศ. 2540 และ 2550 ซึ่งสอดคล้องกับขนาดของเอกสารที่ใหญ่ที่สุด นอกจากนี้ค่า Type-Token Ratio (TTR) ที่วิเคราะห์ด้วย Timeline Chart ที่แบ่งสีตาม `doc_type` พบว่ารัฐธรรมนูญชั่วคราวมักมีค่า TTR สูงกว่า เนื่องจากมีจำนวนมาตราน้อยแต่ใช้คำหลากหลายมากกว่าโดยสัดส่วน

Timeline: How Thai Constitutions Changed Over History

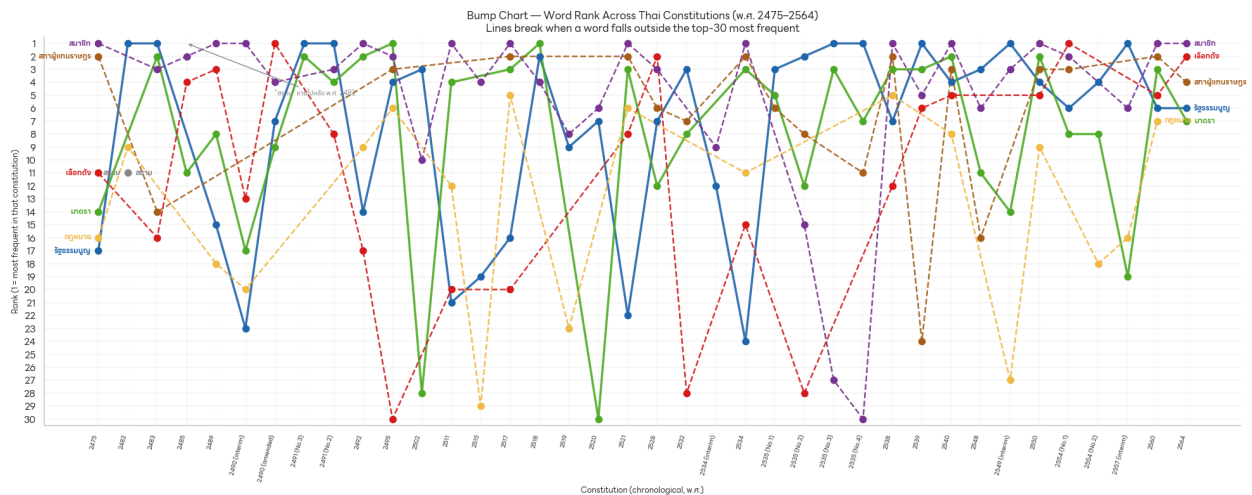


รูปที่ 4.5.1 กราฟแสดงการเติบโตของคลังคำสะสม (Cumulative Vocabulary Growth) เมื่อเพิ่มมาตราทีละมาตราตามลำดับเวลา



รูปที่ 4.5.2 Timeline แสดงค่า Type-Token Ratio ตามลำดับเวลา

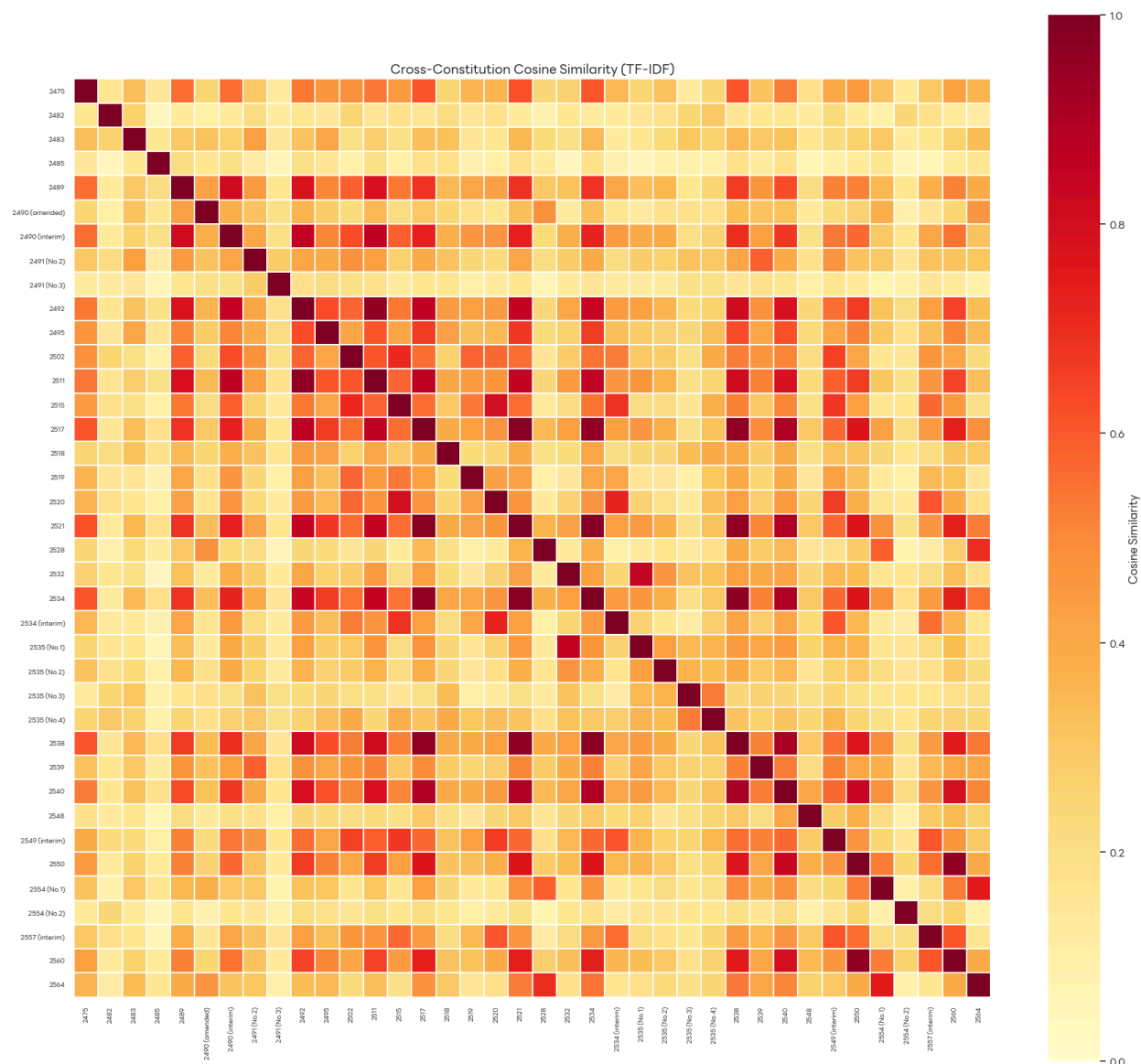
Bump Chart ของคำข้ามรัฐธรรมนูญ แสดงให้เห็นว่าคำกลุ่มหลัก เช่น "รัฐธรรมนูญ" และ "มาตรา" ครองอันดับต้นอย่างสม่ำเสมอตลอดประวัติศาสตร์ ขณะที่คำบางคำเช่น "สยาม" ปรากฏเฉพาะในยุคต้น และ "ประชาธิปไตย" ขึ้นอันดับสูงขึ้นในยุคหลัง



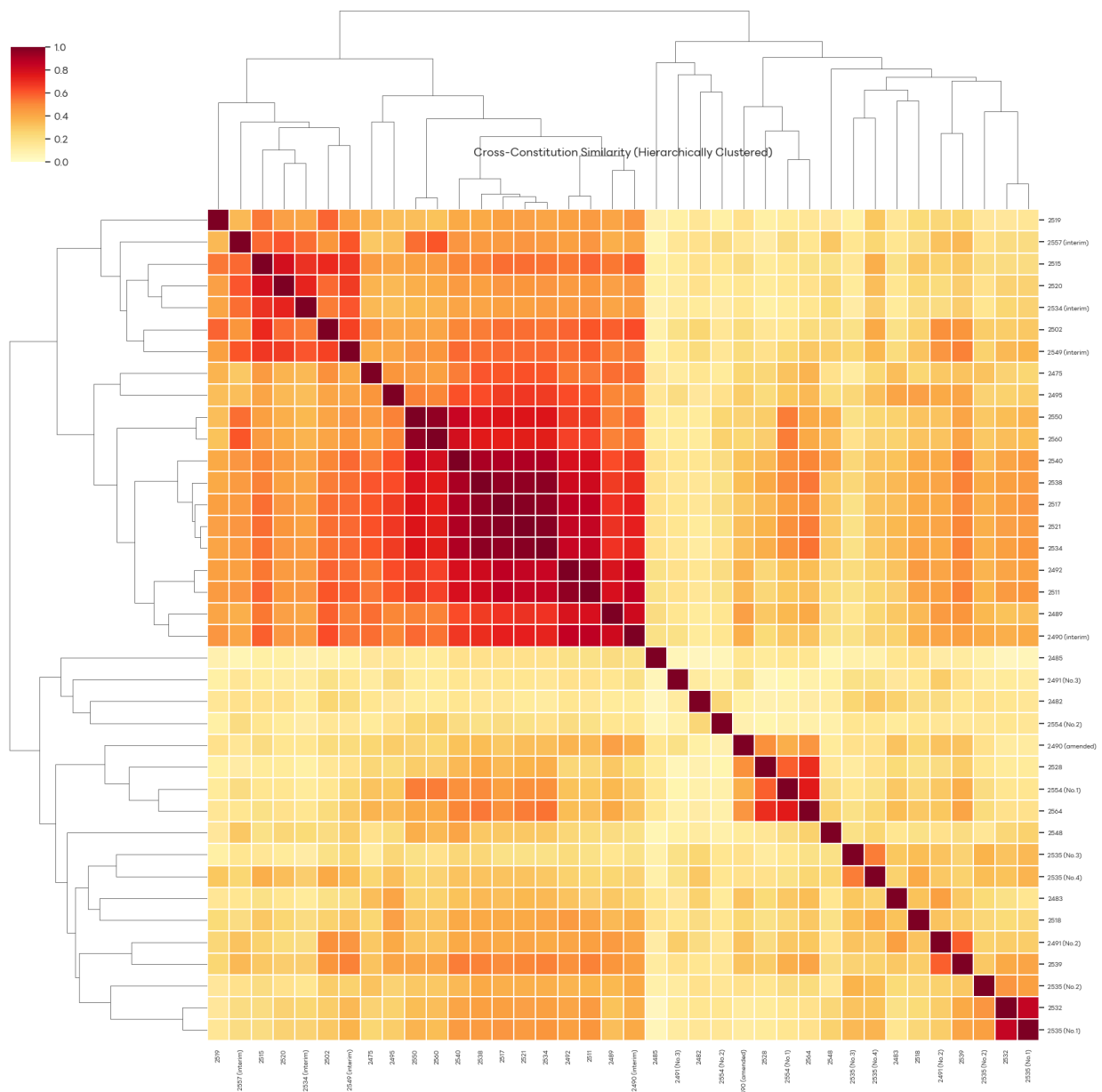
รูปที่ 4.5.3 Bump Chart แสดงการเปลี่ยนแปลงอันดับของ 25 คำยอดนิยมข้ามรัฐธรรมนูญตามลำดับ

4.6. ความคล้ายคลึงระหว่างรัฐธรรมนูญ (Cross-Constitution Similarity)

Heatmap ของ Cosine Similarity 38x38 ฉบับ และ Clustermap แบบ Hierarchical Clustering เผยให้เห็นกลุ่มรัฐธรรมนูญที่มีเนื้อหาใกล้เคียงกันอย่างชัดเจน โดยรัฐธรรมนูญ พ.ศ. 2521, 2534 และ 2538 มีค่าความคล้ายคลึงสูงสุดในกลุ่ม (Similarity > 0.96) บ่งชี้ว่าฉบับหลังมีการคัดลอกหรืออ้างอิงโครงสร้างจากฉบับก่อนหน้าในสัดส่วนสูง ในทางกลับกัน รัฐธรรมนูญ พ.ศ. 2485 มีค่าความคล้ายคลึงกับฉบับอื่นต่ำที่สุด สะท้อนความเป็นเอกลักษณ์ทางเนื้อหาของรัฐธรรมนูญในยุคสงครามโลกครั้งที่ 2



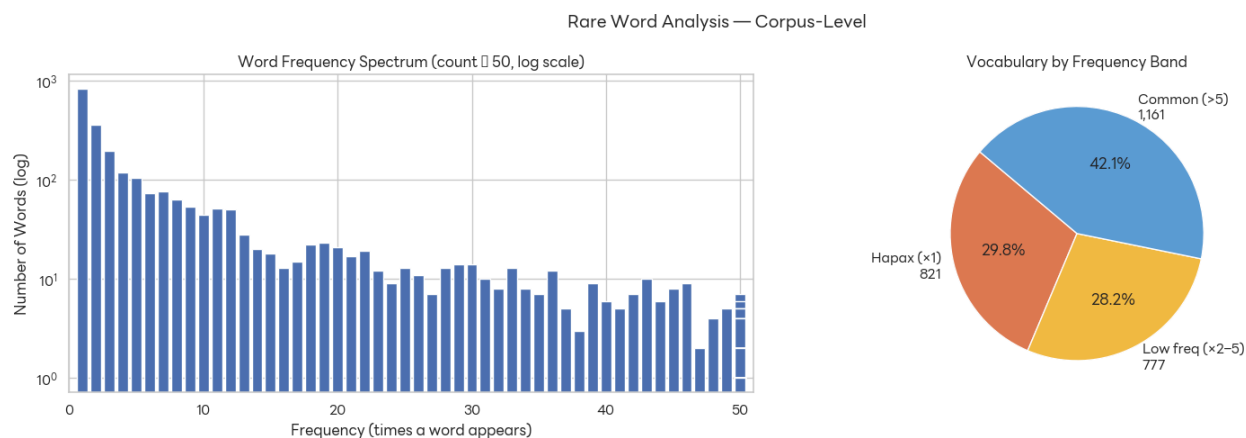
รูปที่ 4.6.2 Heatmap แสดงค่า Cosine Similarity ระหว่างรัฐธรรมนูญ 38 ฉบับ คำนวณจาก TF-IDF Vector



รูปที่ 4.6.2 Clustermap แสดงการจัดกลุ่มรัฐธรรมนูญด้วย Hierarchical Clustering จาก TF-IDF

4.7 การวิเคราะห์คำหายาก (Rare Word Analysis)

จาก Frequency Spectrum ที่ใช้สเกล Log พบว่าคลังคำของรัฐธรรมนูญไทยแสดงพฤติกรรมตามกฎ Zipf ซึ่งเป็นลักษณะสากลของภาษาธรรมชาติ กล่าวคือคำส่วนใหญ่ปรากฏน้อยมากและมีคำเพียงส่วนน้อยที่ปรากฏบ่อย แผนภูมิวงกลมแสดงว่าคำที่ปรากฏเพียงครั้งเดียว (Hapax Legomena) คิดเป็นสัดส่วนสูงสุดในคลังคำ ซึ่งส่วนใหญ่เป็นข้อมูลของชื่อเฉพาะ คำย่อ และศัพท์กฎหมายเฉพาะยุคสมัย การวิเคราะห์คำเด่นเฉพาะรัฐธรรมนูญแต่ละฉบับ (Exclusive Rare Words) ด้วย Interactive Dropdown ช่วยให้ผู้ใช้สามารถระบุวาระและประเด็นเฉพาะที่รัฐธรรมนูญแต่ละฉบับให้ความสำคัญได้อย่างละเอียด



รูปที่ 4.7.1 (ซ้าย) Frequency Spectrum สเกล Log แสดงจำนวนคำที่ปรากฏ N ครั้ง (ขวา) แผนภูมิวงกลมแบ่งคำตามกลุ่มความถี่ (Hapax / Low-freq / Common)

4.8 Visualization อื่น ๆ : Word Cloud

เพื่อสำรวจภาพรวมของคำที่ปรากฏบ่อยภายในชุดข้อมูลเชิงข้อความ ได้มีการสร้าง **Word Cloud** ซึ่งเป็นการแสดงผลเชิงภาพที่ขนาดของคำจะแปรผันตามความถี่ในการปรากฏของคำนั้น ๆ ภายใน corpus ทั้งหมด ช่วยให้สามารถมองเห็นหัวข้อสำคัญ คำหลัก (keywords) หรือแนวโน้มของข้อมูลได้อย่างรวดเร็ว



รูปที่ 4.8.1 แสดง Word Cloud ของคำทั้งหมดภายใน corpus

บทที่ 5 การจำแนกหัวข้อด้วย Topic Modeling

หลังจากผ่านกระบวนการเตรียมข้อมูลและวิเคราะห์เชิงสำรวจในบทที่ 3 และ 4 แล้ว ผู้จัดทำได้นำข้อมูลมาตรวจจาก sections_v2.csv ทั้งหมด 2,797 มาตรา จาก 38 ฉบับ มาผ่านกระบวนการ Topic Modeling เพื่อจัดกลุ่มมาตราตามเนื้อหาเชิงความหมาย โดยไม่ยึดโครงสร้างหมวดเดิมที่ไม่สอดคล้อง กันระหว่างฉบับ

5.1 ที่มาและปัญหาสำคัญ

ปัญหาสำคัญที่นำมาสู่กระบวนการนี้คือรัฐธรรมนูญไทย 38 ฉบับ มีโครงสร้างหมวดที่ไม่สอดคล้องกัน หมวดเดียวกันอาจมีชื่อต่างกันในแต่ละฉบับหรือเนื้อหาเดียวกันอาจถูกจัดอยู่คนละหมวดในรัฐธรรมนูญต่าง ยุคสมัย ดังนั้นการจัดกลุ่มมาตราโดยอาศัย `chapter_title` หรือ `section_title` เพียงอย่างเดียวจึงไม่เพียงพอ สำหรับการวิเคราะห์เชิงเปรียบเทียบข้ามฉบับ

วัตถุประสงค์ของบทนี้จึงมีสองส่วนหลัก ส่วนแรกคือการใช้ Topic Modeling เพื่อจัดกลุ่มมาตรา ตามเนื้อหาเชิงความหมาย (Semantic Content) แทนโครงสร้างหมวดเดิม และส่วนที่สองคือการแสดงให้เห็นว่า มาตราหนึ่งสามารถมีส่วนร่วมในหลาย Theme พร้อมกันได้ (Multi-theme Membership) ซึ่งสะท้อนความ ซับซ้อนของเนื้อหารัฐธรรมนูญที่มักครอบคลุมหลายประเด็นในมาตราเดียว

5.2 การพัฒนาวิธีการ (Methodology Progression)

ได้ทำการทดลองวิธีการทั้งหมด 4 Attempts ก่อนจะได้ Pipeline สุดท้าย โดยแต่ละ Attempt ได้ข้อ เรียนรู้ที่นำไปสู่การปรับปรุงในขั้นต่อไป

Attempt 1 เริ่มจากการทดลอง LDA (Latent Dirichlet Allocation) ซึ่งเป็น Classic Probabilistic Topic Model โดยใช้ CountVectorizer แปลงข้อความเป็น Bag-of-Words Matrix แล้วให้ LDA ค้นหา Latent Topics วิธีนี้มองข้อความเป็นเพียงการนับความถี่ของคำ โดยสมมติว่า Document หนึ่งประกอบขึ้นจากการ ผสมหลาย Topics และแต่ละ Topic คือการกระจายตัวของคำ อย่างไรก็ตามพบว่า LDA ไม่เข้าใจความหมาย เชิง Semantic เลย มองแค่คำไหน Co-occur กันบ่อย Topics ที่ได้จึงเป็นกลุ่มคำที่มักปรากฏด้วยกัน แต่ไม่ สะท้อน Constitutional Theme ที่ชัดเจน ประกอบกับภาษาไทยต้องการ Tokenization พิเศษเพื่อแยกคำ ซึ่ง Bag-of-Words Approach ยิ่งทำให้สูญเสีย Context มากขึ้น บทเรียนจาก Attempt นี้คือจำเป็นต้องใช้วิธีที่ เข้าใจความหมายเชิง Semantic และ Embedding Model ที่เหมาะกับภาษาไทยโดยเฉพาะ

Attempt 2 จึงเปลี่ยนมาใช้ WangchanBERTa ซึ่งเป็น BERT-based Model ที่ถูก Pre-train บน Thai Common Crawl Corpus โดยตรง ร่วมกับ BERTopic แบบ Unsupervised โดยไม่มี Seed Guidance ผลลัพธ์ พบ 79 Topics แต่ยังมีปัญหาหลักสามประการ ได้แก่ มาตรา 617 มาตรา (22.1%) ถูก HDBSCAN จัดเป็น Outlier Cluster ไม่ได้รับ Topic ใดเลย จำนวน 79 Topics มีความละเอียดสูงเกินไปจนยากต่อการตีความใน ระดับ Constitutional Theme และ BERTopic Auto-generated Topic Names เป็นเพียง Top Keywords เช่น 0_ศาลฎีกา_ผู้ดำรงตำแหน่ง_ไต่สวน_ยื่น ซึ่งต้องอาศัย Human Interpretation ทุก Topic และ Scale ไม่ได้กับ 79 Topics

Attempt 3 จึงนำ Domain Knowledge ของโครงสร้างรัฐธรรมนูญไทยมาใช้โดยกำหนด Seed Themes ผ่าน seed_topic_list Parameter ของ BERTopic เริ่มจาก 14 Themes แล้วขยายเป็น 15 Themes โดยเพิ่ม "พรรคการเมืองและการเลือกตั้ง" เพื่อให้ครอบคลุมโครงสร้างรัฐธรรมนูญไทยได้ครบถ้วนขึ้น วิธีนี้ได้ 101 Topics พร้อม Soft Scoring ระดับมาตรา อย่างไรก็ตามพบว่า Soft Scoring ใน Attempt นี้ใช้ Keyword Overlap เป็น Heuristic เพียงอย่างเดียว ทำให้มาตราที่ไม่มี Seed Keyword ปรากฏในข้อความถึง 352 มาตรา (12.8%) ถูกจัดเป็น "อื่น ๆ (emergent)" ทั้งหมดโดยไม่คำนึงถึง Semantic Content ที่แท้จริง

Attempt 4 จึงแก้ปัญหา Emergent Rate ที่สูงนี้ด้วย Hybrid Approach โดยเพิ่ม Embedding-based Fallback สำหรับมาตราที่ Keyword Approach ไม่สามารถ Assign ได้ ซึ่งจะอธิบายละเอียดในหัวข้อ 5.3

ตาราง 11 สรุปการพัฒนา Methodology ใน 4 Attempts

Attempt	วิธีการ	ปัญหาหลัก
1	LDA + Bag-of-Words	ไม่เข้าใจ Semantic ภาษาไทย
2	WangchanBERTa + Unsupervised BERTopic	79 Topics ตีความยาก, 22.1% Outlier
3	Guided BERTopic + 15 Seed Themes	Emergent สูง 12.8%
4	Hybrid Soft Scoring	Emergent ลดเหลือ 0.7%

5.3 รายละเอียด Pipeline (Attempt 4)

5.3.1 Preprocessing

ขั้นตอนแรกของ Pipeline คือการโหลด sections_v2.csv จำนวน 2,797 มาตรา แล้ว Tokenize ด้วย AttaCut ซึ่งเป็น Thai-specific Tokenizer ที่ใช้ Deep Learning Model แยกคำภาษาไทยได้แม่นยำกว่า Rule-based Approaches จากนั้นกรอง Stopwords ออกด้วย 3 กลุ่มที่กำหนดเอง ได้แก่ Group A คือ Function Words ที่ทำหน้าที่ทางไวยากรณ์แต่ไม่มีความหมายเชิง Topic เช่น ตาม, แห่ง, โดย, ซึ่ง, Group B คือ Document Structure Noise ที่ปรากฏในทุกมาตราและจะ Dominate Embedding เช่น มาตรา, หมวด, บรรค, รัฐธรรมนูญ, ราชอาณาจักรไทย และ Group C คือ Legal Boilerplate ที่เป็นคำกฎหมายทั่วไป ซึ่งไม่ช่วยแยกแยะ Theme เช่น กฎหมาย, ระเบียบ, ภายใต, บังคับ

Group B มีความสำคัญเป็นพิเศษ เนื่องจากคำว่า "รัฐธรรมนูญ" ปรากฏในทุกมาตรา หากไม่กรองออก Embedding ของทุกมาตราจะถูกดึงไปยังทิศทางเดียวกันใน Vector Space ทำให้แยกแยะ Theme ไม่ได้เลย หลัง Preprocessing เหลือมาตราที่ใช้งานได้ 2,756 มาตรา โดย 41 มาตราถูก Drop เนื่องจากข้อความว่างหลังการกรอง

5.3.2 Embedding

แต่ละมาตราถูก Embed ด้วย WangchanBERTa (airesearch/wangchanberta-base-att-spm-uncased) ซึ่งเป็น CamemBERT (RoBERTa-based) Architecture ที่ถูก Pre-train บน Thai Common Crawl ขนาด 78GB โดย AIResearch.in.th โมเดลนี้ใช้ SentencePiece Tokenizer ที่ Train บนภาษาไทยโดยเฉพาะ ต่างจาก Multilingual Models ที่ต้องแบ่ง Capacity กับกว่า 100 ภาษา โดย WangchanBERTa ใช้ 768 Dimensions ทั้งหมดสำหรับภาษาไทยเพียงภาษาเดียว

ผลลัพธ์ที่ได้คือ Matrix ขนาด $2,797 \times 768$ โดยแต่ละแถวคือ Vector ที่แทนความหมายของ มาตรา นั้น

5.3.3 Semi-supervised Topic Discovery (Guided BERTopic)

BERTopic ทำงานเป็น Orchestrator ที่เรียก 3 Components ต่อเนื่องกัน โดยเริ่มจาก UMAP สำหรับลด Dimensionality จาก 768 ลงเหลือ 5 Dimensions โดยรักษา Local Semantic Neighborhood ไว้ด้วยการกำหนด `n_neighbors=15` และใช้ `metric='cosine'` เพื่อวัดความคล้ายกันด้วย Angle ใน Vector Space แทน Distance

จากนั้น HDBSCAN ค้นหา Density-based Clusters ใน 5D Space โดยไม่ต้องกำหนดจำนวน Cluster ล่วงหน้า ด้วย `min_cluster_size=12` หมายความว่า Cluster ต้องมีอย่างน้อย 12 มาตรา มิฉะนั้นจะถูกจัดเป็น Outlier (-1) และ `min_samples=3` เพื่อเพิ่มความ Robust ต่อ Noise

ส่วนที่ทำให้ Attempt นี้เป็น Semi-supervised คือการส่ง 15 กลุ่มของ Seed Keywords ที่ Represent แต่ละ Constitutional Theme เข้า BERTopic ผ่าน `seed_topic_list` Parameter BERTopic จะ Embed Seed Keywords เหล่านี้เข้าสู่ Vector Space เดียวกันกับมาตรา แล้วใช้ตำแหน่งของ Seed Words

เป็น Anchor เพื่อดึง Clusters ที่อยู่ใกล้เคียงให้รวมตัวรอบ Theme นั้น ตัวอย่างที่เห็นได้ชัดเจน คือมาตราเกี่ยวกับ "ผู้พิพากษา" สามารถ Cluster กับ Theme ตุลาการได้ แม้คำว่า "ผู้พิพากษา" จะไม่อยู่ใน Seed List ก็ตาม เพราะ WangchanBERTa รู้ว่า "ผู้พิพากษา" และ "ศาล" อยู่ใกล้กันใน Semantic Space ผลลัพธ์ที่ได้คือ 101 Topics ซึ่งถูก Map ไปยัง 15 Seed Themes เหล่านี้

ตาราง 12 Seed Keywords ทั้ง 15 Themes ที่ใช้ใน seed_topic_list

Theme	Seed Keywords
ทั่วไป/โครงสร้าง	บททั่วไป, บทสุดท้าย, บทเฉพาะกาล, แก้ไขเพิ่มเติม
สถาบันพระมหากษัตริย์	พระมหากษัตริย์, สืบราชสมบัติ, องคมนตรี, ผู้สำเร็จราชการ
สิทธิ-เสรีภาพ-หน้าที่	สิทธิ, เสรีภาพ, หน้าที่, ปวงชน, ชนชาวไทย
แนวนโยบายแห่งรัฐ	แนวนโยบาย, พื้นฐานแห่งรัฐ, รัฐ
นิติบัญญัติ	รัฐสภา, สภาผู้แทนราษฎร, วุฒิสภา, ตรากฎหมาย
บริหาร	คณะรัฐมนตรี, นายกรัฐมนตรี, รัฐมนตรี, บริหาร
ตุลาการ	ศาล, ศาลยุติธรรม, ศาลปกครอง, ศาลรัฐธรรมนูญ
องค์กรอิสระ/องค์กรตามรัฐธรรมนูญ	กกต, ป.ป.ช., ผู้ตรวจการแผ่นดิน, สตง, กสม
ตรวจสอบอำนาจรัฐ/ต้านทุจริต	ตรวจสอบ, ผลประโยชน์, บัญชีทรัพย์สิน, ถอดถอน, ทุจริต
การคลังและงบประมาณ	การคลัง, งบประมาณ, เงินแผ่นดิน, ตรวจเงินแผ่นดิน
ท้องถิ่น	ท้องถิ่น, ปกครองส่วนท้องถิ่น
การมีส่วนร่วมทางการเมือง	ประชามติ, มีส่วนร่วม, เข้าชื่อเสนอ, ลงคะแนนโดยตรง
พรรคการเมืองและการเลือกตั้ง	พรรคการเมือง, เลือกตั้ง, ผู้สมัคร, สมาชิกสภา, เขตเลือกตั้ง
ปฏิรูปประเทศ	ปฏิรูปประเทศ
จริยธรรมทางการเมือง	จริยธรรม, มาตรฐาน, ผู้ดำรงตำแหน่งทางการเมือง

5.3.4 Hybrid Soft Scoring

นี่คือส่วนที่แตกต่างจาก Attempt 3 อย่างมีนัยสำคัญ แต่ละมาตราจะผ่านการประเมินสองขั้นตอนตามลำดับ

ขั้นตอนแรกคือ Keyword Overlap ซึ่งเป็น Primary Path ครอบคลุม 2,406 มาตรา (87.3%) สำหรับมาตราที่มี Token ตรงกับ Seed Lexicon อย่างน้อยหนึ่ง Theme คะแนนจะคำนวณจากสัดส่วนของ Seed Keywords ที่ปรากฏในข้อความ แล้ว Normalize ให้รวมเป็น 1.0 ข้ามทุก Theme ที่ได้คะแนน วิธีนี้ให้ความแม่นยำสูงสำหรับภาษากฎหมายรัฐธรรมนูญ เนื่องจากคำศัพท์เฉพาะทางปรากฏอย่างสม่ำเสมอ และชัดเจน เช่น "ศาลรัฐธรรมนูญ" ปรากฏเฉพาะในมาตราตุลาการ และ "คณะรัฐมนตรี" ปรากฏเฉพาะในมาตราบริหาร

ขั้นที่สองคือ Embedding Cosine Similarity ซึ่งเป็น Fallback สำหรับ 331 มาตรา (12.0%) ที่ไม่มี Keyword Match เลย ซึ่งใน Attempt 3 มาตราเหล่านี้จะกลายเป็น Emergent ทั้งหมด วิธีนี้เริ่มจากคำนวณ Theme Centroid โดยเฉลี่ย Embedding ของมาตราทุกมาตราที่ BERTopic Assign ให้ Theme นั้น แล้วคำนวณ Cosine Similarity ระหว่าง Section Embedding กับ Theme Centroids ทั้ง 15 Theme อย่างไรก็ดีตามพบว่า Cosine Similarity ใน 768-dimensional Space มักให้ Distribution ที่ Flat มาก โดยทุก Theme ได้คะแนนอยู่ในช่วง 0.08–0.10 เนื่องจากข้อความรัฐธรรมนูญทั้ง Corpus อยู่ใน Semantic Region เดียวกัน จึงจำเป็นต้องใช้ Temperature-scaled Softmax ด้วย Temperature 0.05 เพื่อ Amplify ความแตกต่างเล็กน้อยให้กลายเป็นความแตกต่างที่ชัดเจน จากนั้นเก็บเฉพาะ Theme ที่ Sharpened Score $\geq$ Threshold แล้ว Normalize ที่เหลือให้รวมเป็น 1.0

มาตราที่ผ่านทั้งสองเส้นทางแล้วยังไม่สามารถ Assign ได้จะถูกจัดเป็น "อื่น ๆ (Emergent)" ซึ่งเหลือเพียง 19 มาตรา (0.7%) มาตราเหล่านี้ไม่มีทั้ง Keyword Evidence และ Semantic Signal ที่แข็งแกร่งต่อ Theme Centroid ใดเลย ส่วนใหญ่เป็นมาตราที่ระบุเพียงชื่อเรียกของรัฐธรรมนูญ หรือมาตราที่อ้างอิงมาตราอื่นโดยไม่มีเนื้อหาในตัวเอง เมื่อเปรียบเทียบกับ Attempt 3 ที่มี Emergent ถึง 352 มาตรา (12.8%) ความแตกต่างนี้แสดงให้เห็นว่า Emergent ใน Attempt 4 เป็นมาตราที่ Genuinely ไม่มีธีมจริง ๆ ไม่ใช่ผลจากข้อจำกัดของ Keyword List อีกต่อไป

ตาราง 13 การกระจายตัวของจำนวน Theme ต่อมาตรา (Attempt 4)

	Attempt 3	Attempt 4
Keyword Path	2,406 มาตรา (87.3%)	2,406 มาตรา (87.3%)
Embedding Fallback	—	331 มาตรา (12.0%)
Emergent (อื่น ๆ)	352 มาตรา (12.7%)	19 มาตรา (0.7%)

ผลสำคัญที่สุดของ Hybrid Scoring คือมาตราหนึ่งจะได้รับ Fractional Scores ข้าม Theme พร้อมกัน สะท้อนความจริงที่ว่ามาตรารัฐธรรมนูญมักครอบคลุมหลายประเด็นในมาตราเดียว โดยมาตรา ส่วนใหญ่ได้ 1–2 Themes ซึ่งสะท้อนว่า Keyword Path ให้ผลที่ Focused ส่วนมาตราที่ได้ 3 Themes ขึ้นไปมักเป็นมาตราที่มีเนื้อหาหลายประเด็นจริง หรืออยู่ใน Embedding Path ที่ Distribution กระจาย มากกว่า

ตาราง 14 การกระจายตัวของจำนวน Theme ต่อมาตรา (Attempt 4)

จำนวน Theme ต่อมาตรา	จำนวนมาตรา
1	1,090
2	927
3	512
4	118
5 ขึ้นไป	90

5.4 ผลลัพธ์จากการทำปฏิบัติการ

5.4.1 ไฟล์ Output

Pipeline ใน Attempt 4 ผลิต Output หลักที่ใช้ในการวิเคราะห์ต่อคือ section_theme_scores.csv ซึ่งเก็บ Soft Theme Scores ระดับมาตรา โดยแต่ละ Row คือ (section, theme, score) และ Score รวมต่อ Section = 1.0

5.4.2 ความเสถียรของ Theme (Theme Stability)

ผู้จัดทำวัด Stability ของแต่ละ Theme ด้วย **stable_score** ซึ่งคำนวณจากสัดส่วน 70% ของ Coverage Ratio (จำนวนรัฐธรรมนูญที่มี Theme นี้ปรากฏ) บวกกับ 30% ของ Average Theme Score โดย Theme ที่ปรากฏในรัฐธรรมนูญครบทุกฉบับและมีคะแนนสูงสม่ำเสมอจะได้คะแนน Stability สูงสุด

จากตาราง 15 พบว่า Theme บริหารมี **stable_score** สูงสุดที่ 0.839 และปรากฏในรัฐธรรมนูญครบ ทั้ง 38 ฉบับ (100%) สะท้อนว่าการจัดโครงสร้างฝ่ายบริหารเป็นองค์ประกอบพื้นฐานของรัฐธรรมนูญทุกฉบับ โดยไม่ขึ้นกับยุคสมัยหรือระบอบการเมือง, เริ่มสถาบันพระมหากษัตริย์และแนวนโยบายแห่งรัฐตามมาด้วย stable score 0.838 และ 0.820 ตามลำดับ โดยปรากฏใน 36 จาก 38 ฉบับ (94.7%) ในขณะที่มีปฏิรูปประเทศมีเพียง 2 มาตราใน 2 ฉบับ สะท้อนว่าเป็น Theme ที่เกิดขึ้นเฉพาะในรัฐธรรมนูญยุคหลังตั้งแต่ พ.ศ. 2560 เป็นต้นไป

ตาราง 15 สรุปผลการทำ Theme Stability ทั้งหมด 15 Themes

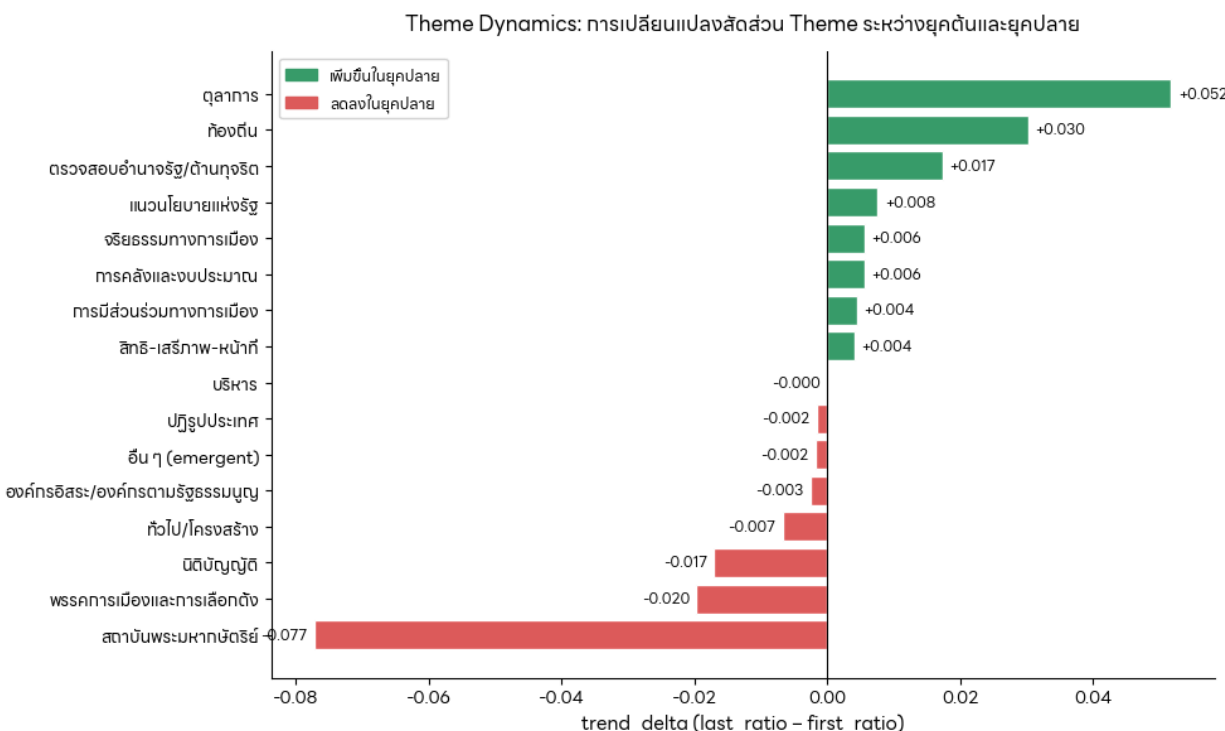
Theme	จำนวนมาตรา	รัฐธรรมนูญ	Coverage	stable_score
บริหาร	730	38/38	100.0%	0.839
สถาบันพระมหากษัตริย์	741	36/38	94.7%	0.838
แนวนโยบายแห่งรัฐ	612	36/38	94.7%	0.820
นิติบัญญัติ	1,257	33/38	86.8%	0.779
พรรคการเมืองและการเลือกตั้ง	428	28/38	73.7%	0.628
สิทธิ-เสรีภาพ-หน้าที่	662	25/38	65.8%	0.598
ตุลาการ	427	22/38	57.9%	0.567
ท้องถิ่น	178	15/38	39.5%	0.406
จริยธรรมทางการเมือง	98	13/38	34.2%	0.373
การคลังและงบประมาณ	91	14/38	36.8%	0.365

ตรวจสอบอำนาจรัฐ/ด้านทุจริต	137	14/38	36.8%	0.352
ทั่วไป/โครงสร้าง	31	10/38	26.3%	0.311
การมีส่วนร่วมทางการเมือง	46	10/38	26.3%	0.289
องค์กรอิสระ/องค์กรตามรัฐธรรมนูญ	19	7/38	18.4%	0.249
ปฏิรูปประเทศ	2	2/38	5.3%	0.337

5.4.3 พลวัตของ Theme (Theme Dynamics)

เพื่อวิเคราะห์ว่า Theme ใดมีน้ำหนักเพิ่มขึ้นหรือลดลงในรัฐธรรมนูญยุคหลัง ผู้จัดทำคำนวณ trend_delta โดยเปรียบเทียบสัดส่วน Theme Mass ระหว่างรัฐธรรมนูญยุคต้น (Percentile 0–35) กับยุคปลาย (Percentile 65–100) ค่าบวกหมายความว่า Theme มีสัดส่วนเพิ่มขึ้นในรัฐธรรมนูญยุคหลัง ส่วนค่าลบหมายความว่าลดลง

จากรูป 5.4.3.1 พบว่า Theme ตุลาการมี trend_delta สูงสุดที่ +0.052 โดยเพิ่มจาก 6.4% ในยุคต้น เป็น 11.6% ในยุคปลาย Theme ท้องถิ่นและตรวจสอบอำนาจรัฐก็เพิ่มขึ้นเช่นกัน สะท้อนพัฒนาการ ของสถาบันทางการเมืองในทิศทางของ Rule of Law ที่ให้ความสำคัญกับการตรวจสอบ ถ่วงดุลอำนาจมากขึ้น ในทางกลับกัน Theme สถาบันพระมหากษัตริย์มี trend_delta ลบมากที่สุดที่ -0.077 โดยลดจาก 19.7% เป็น 12.0% ซึ่งไม่ได้หมายความว่าความสำคัญของสถาบันลดลง แต่สะท้อนว่าเนื้อหาของรัฐธรรมนูญยุคหลัง ขยายครอบคลุมประเด็นอื่น ๆ มากขึ้น ทำให้สัดส่วนโดยรวมลดลงตามธรรมชาติ



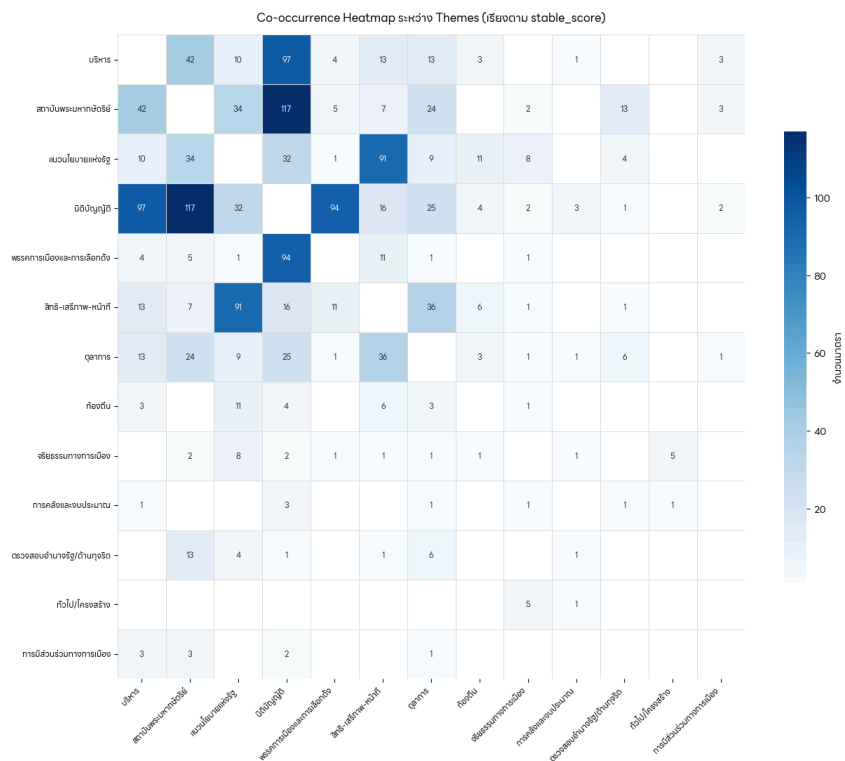
รูปที่ 5.4.3.1 แสดงการเปลี่ยนแปลงสัดส่วน Theme ระหว่างยุคต้นและยุคปลาย

5.4.4 Co-occurrence ระหว่าง Theme

จากตาราง 17 พบว่าคู่ที่มี Co-occurrence สูงสุดคือ นิติบัญญัติ กับ สถาบันพระมหากษัตริย์ ที่ปรากฏร่วมกันใน 117 มาตรา สะท้อนให้เห็นว่ามาตราเกี่ยวกับ Legislative Process ในรัฐธรรมนูญไทยมักกล่าวถึงพระราชอำนาจในกระบวนการนิติบัญญัติควบคู่กันเสมอ เช่น การทรงลงพระปรมาภิไธย หรือการยับยั้งร่างกฎหมาย คู่ลำดับถัดมาคือ นิติบัญญัติ กับ บริหาร (97 มาตรา) และ นิติบัญญัติ กับ พรรคการเมืองและการเลือกตั้ง (94 มาตรา) ซึ่งล้วนสะท้อนให้เห็นว่า Theme นิติบัญญัติเป็น Theme ที่มีการเชื่อมโยงสูงสุดกับ Theme อื่น ๆ ในรัฐธรรมนูญไทย

ตาราง 16 Theme Co-occurrence ที่พบบ่อยที่สุด

Theme A	Theme B	จำนวนมาตรา
นิติบัญญัติ	สถาบันพระมหากษัตริย์	117
นิติบัญญัติ	บริหาร	97
นิติบัญญัติ	พรรคการเมืองและการเลือกตั้ง	94
สิทธิ-เสรีภาพ-หน้าที่	แนวนโยบายแห่งรัฐ	91
บริหาร	สถาบันพระมหากษัตริย์	42
ตุลาการ	สิทธิ-เสรีภาพ-หน้าที่	36
สถาบันพระมหากษัตริย์	แนวนโยบายแห่งรัฐ	34
นิติบัญญัติ	แนวนโยบายแห่งรัฐ	32



รูปที่ 5.4.4.1 แสดง Co-occurrence Heatmap

5.4.5 สรุปผล

จากการหาผลลัพธ์ปฏิบัติการ พบว่าข้อมูลผลลัพธ์ที่สำคัญที่สุดของกระบวนการทั้งหมดในบทนี้คือ ไฟล์ `section_theme_scores` ซึ่งเก็บ Soft Theme Score ของมาตราทุกมาตราในคลังข้อมูล การที่มาตราแต่ละมาตรามี Score กระจาย ข้าม Theme พร้อมกันแทนที่จะได้รับ Label เดียว ทำให้ข้อมูลชุดนี้เปิดโอกาสในการวิเคราะห์ที่ได้สองทิศทาง

ทิศทางที่ 1 ดูว่ามาตราหนึ่งพูดถึงเรื่องอะไร และ Theme นั้นเชื่อมโยงไปหาเรื่องอะไรบ้าง กรองด้วย `section_id` เพื่อดู Score ของแต่ละ Theme ว่ามาตรานั้นพูดถึงประเด็นใดเป็นหลักและรอง หรือกรองด้วย Theme เพื่อเปรียบเทียบความครอบคลุมข้ามรัฐธรรมนูญแต่ละฉบับ

ทิศทางที่ 2 หาความสัมพันธ์ระหว่างมาตรา Theme และ Parameter อื่น ๆ Join กับ `sections_v2.csv` ผ่าน `section_id` เพื่อนำ Parameter อย่าง `era`, `regime_type`, `doc_type`, `section_role` และ `change_mode` มาวิเคราะห์ร่วม เช่น Theme ตุลาการสูงกว่าในยุค Civilian หรือ Military มาตรา amended กระจุกอยู่ใน Theme ไດ

บทที่ 6 การเผยแพร่และการเข้าถึงข้อมูล (Data Publishing & Public Access)

หลังจากผ่านกระบวนการเตรียมข้อมูลและวิเคราะห์เชิงสำรวจแล้ว ผู้จัดทำได้เผยแพร่ผลลัพธ์ในสองรูปแบบหลัก ได้แก่ (1) ชุดข้อมูลสาธารณะบนแพลตฟอร์ม Kaggle เพื่อรองรับการนำไปใช้งานต่อในเชิงวิจัย และ (2) เว็บแอปพลิเคชัน Dashboard แบบโต้ตอบสำหรับสาธารณะทั่วไป โดยมีซอร์สโค้ดเปิดเผยบน GitHub

6.1 ชุดข้อมูลสาธารณะบน Kaggle

ผู้จัดทำได้เผยแพร่ชุดข้อมูลภายใต้ชื่อ **"Twenty Constitutions Digitization"** บนแพลตฟอร์ม Kaggle ซึ่งเป็นชุมชนนักวิทยาศาสตร์ข้อมูลและนักวิจัยที่ใหญ่ที่สุดแห่งหนึ่งของโลก ชุดข้อมูลประกอบด้วยไฟล์หลักสองไฟล์ ได้แก่ `preprocessed_data.csv` (2,797 แถว 21 คอลัมน์ ครอบคลุมรัฐธรรมนูญ 38 ฉบับ ตั้งแต่ พ.ศ. 2475–2564) และ `section_theme_scores.csv` (5,479 แถว ผลการจัดหมวดหมู่เชิงหัวข้อ 13 ด้าน) เพื่อให้ นักวิจัย นักศึกษา และผู้สนใจสามารถดาวน์โหลดและนำไปวิเคราะห์ต่อได้อย่างอิสระ

The screenshot shows the Kaggle dataset page for "Twenty Constitutions Digitization". At the top, it indicates the dataset was updated 2 hours ago. The title "Twenty Constitutions Digitization" is prominently displayed, followed by a subtitle: "38 Thai constitutional documents spanning from 1932 (B.E. 2475) to 2021 (B.E)". Below the title, there are tabs for "Data Card", "Code (0)", "Discussion (0)", and "Suggestions (0)". The "Data Card" tab is selected, showing an "About Dataset" section with a "Dataset Summary" that describes the data as section-level text from 38 Thai constitutional documents, covering full constitutions, interim charters, and amendments. It also mentions support for NLP and political science research. To the right of the summary, there are sections for "Usability" (5.29), "License" (MIT), "Expected update frequency" (Not specified), and "Tags". Below the summary is a "Files Overview" table:

File	Rows	Columns	Description
<code>sections_v2_fixed.csv</code>	2,797	19	Raw section-level text for all constitutional documents
<code>preprocessed_data.csv</code>	2,797	21	Preprocessed version with tokenized text and word counts
<code>section_theme_scores.csv</code>	5,188	6	Theme classification scores per section (semi-supervised)

Below the table, there is a "View more" link. At the bottom, there is a "Data Explorer" section showing a preview of the `exp_topic_assignments.csv` file (1.01 MB), with options to view details, compact, or column view. The preview shows 10 of 15 columns.

รูปที่ 6.1.1 หน้า Dataset Card ของชุดข้อมูล "Twenty Constitutions Digitization" บน Kaggle

ชุดข้อมูลดังกล่าวสามารถเข้าถึงได้ที่ [Kaggle](#)

โครงสร้างของ Dataset Card ที่เผยแพร่ประกอบด้วย

- **ชื่อและคำอธิบายชุดข้อมูล** อธิบายบริบทโครงการ ขอบเขตเวลา และวิธีการเตรียมข้อมูล
- **ไฟล์ข้อมูลหลัก** `preprocessed_data.csv` และ `section_theme_scores.csv` พร้อม Data Dictionary
- **Metadata ทางสถิติ** ได้แก่ จำนวนแถว คอลัมน์ ขนาดไฟล์ และการกระจายตัวของตัวแปรหลัก
- **Usability Score** และ License ที่กำหนดเงื่อนไขการใช้งาน

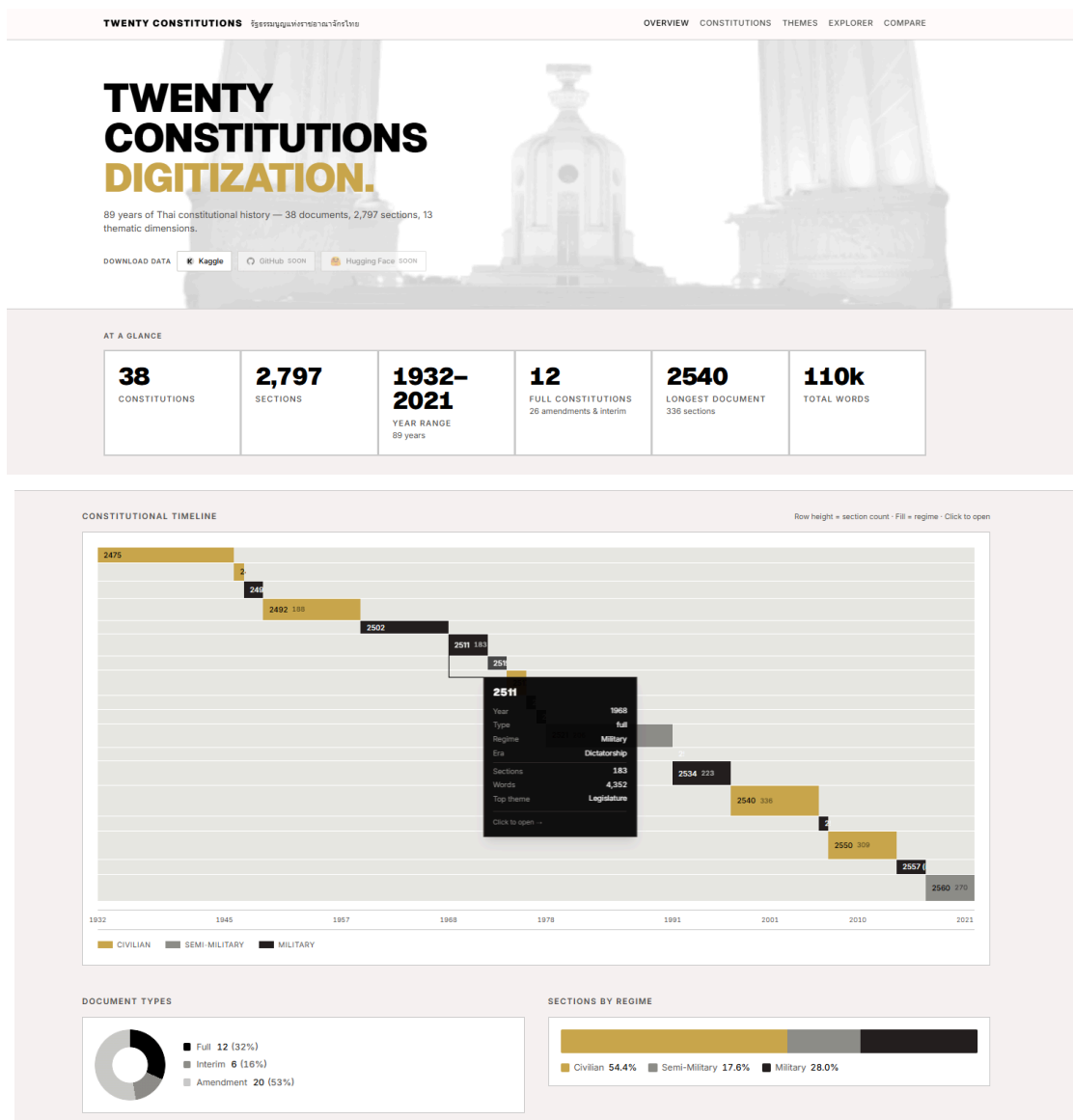
การเผยแพร่บน Kaggle ช่วยให้ชุดข้อมูลเข้าถึงได้ผ่าน Kaggle Notebooks โดยตรงโดยไม่ต้องดาวน์โหลด รวมถึงรองรับการนำไปใช้ใน Machine Learning Pipeline ผ่าน Kaggle API

6.2 เว็บแอปพลิเคชัน Dashboard (Twenty Constitutions Web)

นอกเหนือจากชุดข้อมูลดิบ ผู้จัดทำได้พัฒนาเว็บแอปพลิเคชันแบบโต้ตอบสำหรับเผยแพร่ข้อมูลในรูปแบบที่เข้าถึงได้ง่ายสำหรับสาธารณะทั่วไป โดยไม่จำเป็นต้องมีความรู้ด้านการวิเคราะห์ข้อมูล

ซอร์สโค้ดของโครงการเปิดเผยบน GitHub ที่: [Github](#)
สามารถใช้งานได้ที่นี่: [Link](#)

แอปพลิเคชันพัฒนาด้วย React Router v7 (SSR) บน TypeScript พร้อม Tailwind CSS v4 โดยมีหน้าเว็บดังนี้



รูปที่ 6.2.1 หน้าหลักของเว็บไซต์ โดยมี Overview Dashboard ที่แสดง Constitutional Timeline

ข้อมูลที่ใช้ในการพัฒนา

[1] สำนักวิชาการ สำนักงานเลขาธิการสภาผู้แทนราษฎร, “คลังสารสนเทศรัฐสภา: รัฐธรรมนูญและธรรมนูญการปกครองของประเทศไทย,” 2566. [Online]. เข้าถึงจาก: <https://prt.parliament.go.th/>

แหล่งอ้างอิงทางทฤษฎีและเครื่องมือ

[2] G. Maarten, F. Poli, & J. Dean, “BERTopic: Neural topic modeling with a class-based TF-IDF procedure,” arXiv preprint, 2022. [Online].

เข้าถึงจาก: <https://arxiv.org/abs/2203.05794>

[3] BERTopic Documentation, “BERTopic: Leveraging BERT and c-TF-IDF to create easily interpretable topics,” 2024. [Online].

เข้าถึงจาก: <https://maartengr.github.io/BERTopic/>

[4] L. Lowphansirikul, C. Polpanumas, N. Janpuangtong, & S. Nutanong, “WangchanBERTa: Pretraining transformer-based Thai Language Models,” arXiv preprint, 2021. [Online].

เข้าถึงจาก: <https://arxiv.org/abs/2101.09635>

[5] P. Chormai, P. Tumnongrat, T. Thongtan, & S. Nutanong, “AttaCut: A Fast and Accurate Neural Thai Word Segmenter,” in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020. [Online].

เข้าถึงจาก: <https://aclanthology.org/2020.acl-main.763>

[6] F. Schroff, D. Kalenichenko, & J. Philbin, “Typhoon OCR: Open Vision-Language Model For Thai Document Extraction,” arXiv preprint, 2021. [Online].

เข้าถึงจาก: <https://arxiv.org/abs/2601.14722>

[7] PyMuPDF (fitz) Documentation, “PyMuPDF: Python Bindings for MuPDF,” 2024. [Online].

เข้าถึงจาก: <https://pymupdf.readthedocs.io/>

[8] pdfplumber Documentation, “pdfplumber: Plumb a PDF for detailed information about each text character, rectangle, and line,” 2024. [Online].

เข้าถึงจาก: <https://github.com/jsvine/pdfplumber>

[9] L. McInnes, J. Healy, & J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” arXiv preprint, 2018. [Online].

เข้าถึงจาก: <https://arxiv.org/abs/1802.03426>

[10] R. J. G. B. Campello, D. Moulavi, & J. Sander, “Density-based clustering based on hierarchical density estimates,” in *Advances in Knowledge Discovery and Data Mining*. Springer, 2013. [Online].

เข้าถึงจาก: https://link.springer.com/chapter/10.1007/978-3-642-37456-2_14

[11] G. Salton & C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988. [Online].

เข้าถึงจาก: <https://www.sciencedirect.com/science/article/pii/0306457388900210>

[12] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online].

เข้าถึงจาก: <https://jmlr.org/papers/v12/pedregosa11a.html>

[13] PyThaiNLP Development Team, “PyThaiNLP: Thai Natural Language Processing in Python,” 2024. [Online].

เข้าถึงจาก: <https://pythainlp.github.io/>

ข้อมูลอ้างอิงเพิ่มเติม

[14] Constitue Project, “Constitute: Explore the World's Constitutions,” Comparative Constitutions Project, 2024. [Online].

เข้าถึงจาก: <https://www.constituteproject.org/topics>